

FIAP ON

05

INTERFACES INTELIGENTES: CONECTANDO IA AO USUÁRIO COM REACT E JWT

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Contador básico com useState 8
Figura 2 - Import do useState 9
Figura 3 - Função do Contador 9
Figura 4 - Estado inicial do Contador 9
Figura 5 - Estrutura JSX do Contador 10
Figura 6 - Chamando o Contador no App.jsx 11
Figura 7 - Executando o projeto 11
Figura 8 - Exemplo de uso do useEffect 12
Figura 9 - Importação do hook 13
Figura 10 - Declaração de Estado 13
Figura 11 - Uso do useEffect 13
Figura 12 - Render do jsx 14
Figura 13 - Chamando o componente na tela inicial do App 15
Figura 14 - Criando o temaContexto 16
Figura 15 - Criando o ComponenteComTema 17
Figura 16 - Chamando o componente na página inicial 17
Figura 17 - Estilizando o App 18
Figura 18 - Chamando o componente na tela inicial 18
Figura 19 - Switch para troca de tema 19
Figura 20 - Operador ternário para verificação de condição 19
Figura 21 - Hook reducer 20
Figura 22 - Hook e estado inicial 20
Figura 23 - Função Reducer 21
Figura 24 - Função ContadorReducer 21
Figura 25 - JSX do contador 22
Figura 26 - Chamando o componente Reducer na página inicial 22
Figura 27 - Criando o contexto global 24
Figura 28 - Importando os Hooks e criando o contexto 25
Figura 29 - Componente Provider 25
Figura 30 - Estados e Funções do Contador 26
Figura 31 - JSX do Contador 26
Figura 32 - ComponenteContadorGlobal 27
Figura 33 - Chamando o ComponenteContadorGlobal na página inicial 28
Figura 34 - Hook Fetch API 31
Figura 35 - Chamando o Hook Fetch API na página inicial 34
Figura 36 - Visualização no browser do Hook Fetch API 34
Figura 37 - Hook Axios API 35
Figura 38 - Código do componente principal 38
Figura 39 - Visualização no browser do Hook Axios API 38
Figura 40 - Gerenciamento de Estado 39
Figura 41 - Chamando o Gerenciamento de Estado na página inicial 42
Figura 42 - Criando o componente de gerador de respostas com IA 44
Figura 43 - Chamando o componente de gerador de respostas com IA na página inicial 47
Figura 44 - Criando o componente de autenticação com Context API 51
Figura 45 - Criando o componente de painel de login 54
Figura 46 - Criando o componente de Área Restrita 56

Figura 47 - Chamando Login e Área Restrita na página inicial 58
Figura 48 - Tela de login no navegador.....59

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Importação do hook useEffect 31
Código-fonte 2 - Declaração do componente FetchExemplo 31
Código-fonte 3 - Inicialização do useEffect 31
Código-fonte 4 - Requisição HTTP com fetch 32
Código-fonte 5 - Conversão da resposta em JSON 32
Código-fonte 6 - Exibição de dados no console 32
Código-fonte 7 - Exibindo console error 32
Código-fonte 8 - Dependência inicializada vazia no useEffect 33
Código-fonte 9 - Render do JSX 33
Código-fonte 10 - Importação de recursos do React 36
Código-fonte 11 - Declaração do componente AxiosExemplo 36
Código-fonte 12 - Início de efeitos com o hook useEffect 36
Código-fonte 13 - Uso do axios para requisição POST 36
Código-fonte 14 - Tratamento de resposta no console log 37
Código-fonte 15 - Tratamento de erros com console error 37
Código-fonte 16 - Render JSX da API com axios 37
Código-fonte 17 - Importação dos hooks useEffect e useState 40
Código-fonte 18 - Declaração da função do componente 40
Código-fonte 19 - Constantes de estados locais 40
Código-fonte 20 - Carregando dados da API 40
Código-fonte 21 - Requisição com fetch API 41
Código-fonte 22 - Feedback de requisição 41
Código-fonte 23 - Render da requisição com feedback ao usuário 42
Código-fonte 24 - Retorno visual ao usuário no browser 43
Código-fonte 25 - Código fonte da imagem para testes 45
Código-fonte 26 - Declaração de estados usando hooks 46
Código-fonte 27 - Resposta assíncrona de requisição com axios 46
Código-fonte 28 - Exibição de erro na tela 47
Código-fonte 29 - Importação de hooks e context do React 51
Código-fonte 30 - Exportando contexto de autenticação 51
Código-fonte 31 - Passando a autenticação entre componentes 52
Código-fonte 32 - Gerenciamento de estado iniciado como nulo 52
Código-fonte 33 - Simulando login se tiver token 52
Código-fonte 34 - Simulando token para login 53
Código-fonte 35 - Removendo token ao sair 53
Código-fonte 36 - Passando dados via context entre componentes 53
Código-fonte 37 - Importação de autenticação do React 54
Código-fonte 38 - Iniciando sessão ao receber a autenticação 54
Código-fonte 39 - Renderização condicional baseada na autenticação 55
Código-fonte 40 - Importação da autenticação 56
Código-fonte 41 - Recebendo dados após a autenticação 56
Código-fonte 42 - Feedback de falha de login 57
Código-fonte 43 - Exibição de conteúdo após tela autenticada 57
Código-fonte 44 - Mock para login 57
Código-fonte 45 - Exibição de conteúdo após autenticado 58

FIAP

SUMÁRIO

1 INTERFACES INTELIGENTES: CONECTANDO IA AO USUÁRIO COM REACT E JWT 6
1.1 Objetivos de Aprendizagem 6
1.2 Aplicações Práticas no Mercado de Software 7

2 HOOKS AVANÇADOS NO REACT 8
2.1 useState 8
2.1.1 Exemplo de uso do useState – Criando um contador simples 8
2.2 useEffect 12
2.2.1 Exemplo de uso do useEffect – Criando um relógio digital que atualiza sozinho 12
2.3 useContext 15
2.3.1 Exemplo de uso do useContext – Compartilhando dados entre componentes sem props 16
2.4 useReducer 19
2.4.1 Exemplo de uso do useReducer – Criando um contador com lógica mais controlada 20

3 GERENCIAMENTO GLOBAL DE ESTADO COM CONTEXT API 24
3.1 Criando providers e consumers 24
3.1.1 Criando o contexto global 24
3.1.2 Consumindo o contexto em um componente 27
3.2 Evitando o “prop drilling” com context 28
3.3 Boas práticas e limitações 29
3.4 Alternativas modernas: Redux e zustand (visão geral) 29

4 INTEGRAÇÃO COM APIS DE INTELIGÊNCIA ARTIFICIAL 30
4.1 O papel das APIs de IA em aplicações modernas 30
4.2 Consumo de APIs com fetch e axios 30
4.2.1 Exemplo com fetch 30
4.2.2 Exemplo com axios 35
4.3 Gerenciamento de estados de carregamento e erro 39
4.4 Exemplo prático: Gerador de respostas com IA 43

5 AUTENTICAÇÃO DE USUÁRIOS COM JWT E CONTEXT API 50
5.1 O que é o JSON Web Token (JWT)? 50
5.2 Armazenamento e verificação de tokens 50
5.3 Implementando autenticação com Context API 50
5.4 Proteção de rotas e controle de acesso 56
5.5 Simulação de backend com mock 57

REFERÊNCIAS 60

1 INTERFACES INTELIGENTES: CONECTANDO IA AO USUÁRIO COM REACT E JWT

Neste capítulo, aprofundaremos o uso do ReactJS com foco em recursos mais avançados que tornam a biblioteca ainda mais poderosa e adequada para aplicações modernas. Compreenderemos como os **Hooks** mais sofisticados permitem lidar com lógica de estado complexa, como a Context API torna possível o compartilhamento eficiente de dados em toda a aplicação e como padrões arquiteturais melhoram a escalabilidade dos projetos.

Adicionalmente, exploraremos o consumo de APIs de Inteligência Artificial no frontend, ampliando a capacidade da aplicação de interagir com sistemas externos inteligentes, como assistentes virtuais, reconhecimento de imagem ou análise de sentimentos. Por fim, abordaremos autenticação segura com JWT e boas práticas na componentização de interfaces.

1.1 Objetivos de Aprendizagem

Ao final deste capítulo, você será capaz de:

- Compreender e aplicar Hooks avançados como useEffect, useContext e useReducer para controlar a lógica de efeitos, o compartilhamento de estado e os fluxos complexos.
- Utilizar a **Context API** para gerenciar estado global sem recorrer a bibliotecas externas.
- Integrar aplicações React com **APIs de Inteligência Artificial** para exibir dados inteligentes e personalizar a experiência do usuário.
- Implementar **autenticação com JWT** utilizando a Context API e simulação de backend local.
- Organizar a arquitetura do frontend utilizando **padrões modernos de componentização**, como o **Atomic Design**, visando a legibilidade, reutilização e escalabilidade.

- Desenvolver aplicações reativas, seguras e integradas com serviços externos, prontas para os desafios do mercado de software moderno.

1.2 Aplicações Práticas no Mercado de Software

O domínio dos temas abordados neste capítulo irá capacitá-lo a desenvolver sistemas com características cada vez mais exigidas pela indústria de tecnologia, como:

- **Plataformas com experiência personalizada:** como sistemas de recomendação, assistentes inteligentes e dashboards que se adaptam ao comportamento do usuário.
- **Aplicações seguras e escaláveis:** utilizando JWT para autenticação e boas práticas de arquitetura para facilitar manutenção e crescimento do sistema.
- **Soluções com integração à IA:** cada vez mais demandadas em áreas como saúde, varejo, educação, finanças e marketing, onde a exibição de insights extraídos de dados em tempo real se torna um diferencial competitivo.
- **Ambientes de desenvolvimento colaborativo:** padrões como Atomic Design facilitam o trabalho em equipe, permitindo que desenvolvedores, designers e analistas atuem de forma coesa e produtiva.

Ao dominar esses recursos, você, aluno, aproxima-se da prática profissional e passa a estar apto para compor times de desenvolvimento frontend em empresas que demandam soluções modernas, conectadas e inteligentes.

2 HOOKS AVANÇADOS NO REACT

Vamos continuar o projeto iniciado anteriormente, estruturado com React e Vite. Agora, organizaremos os arquivos em uma nova pasta chamada **src/hooks-examples/**.

Essa pasta irá conter todos os arquivos desta aula de aprofundamento em Hooks, mantendo o código modular e reutilizável. Cada exemplo será uma nova abordagem de controle de estado, efeito ou contexto.

2.1 useState

O **useState** é um Hook do React que permite criar e atualizar valores internos de um componente. Ele é utilizado quando precisamos que o componente "lembre" de um valor ao longo do tempo, como contadores, campos de formulário ou status de exibição. Cada vez que o estado muda, o React re-renderiza o componente com o novo valor.

2.1.1 Exemplo de uso do useState – Criando um contador simples

Crie um arquivo em "**src/hooks-examples/ContadorBasico.jsx**" e escreva o código conforme a figura:

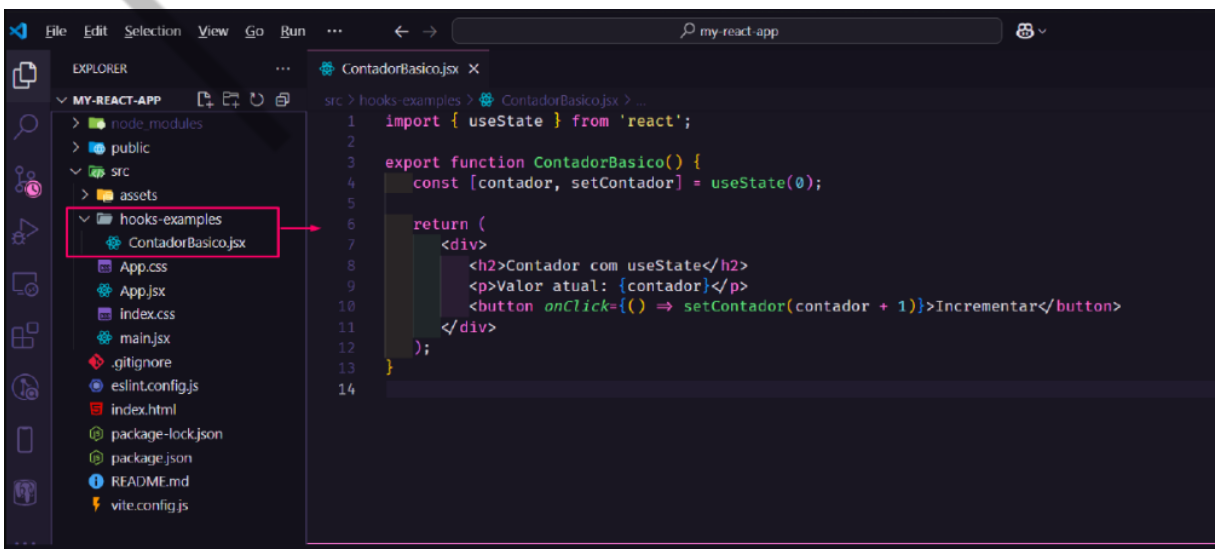


Figura 1 - Contador básico com useState
Fonte: Elaborado pelo autor (2025)

Explicação dos trechos do código:

```
import { useState } from 'react';
```

Figura 2 - Import do useState
Fonte: Elaborado pelo autor (2025)

Neste trecho, estamos importando o Hook useState da biblioteca React. Esse Hook é usado para declarar variáveis de **estado** dentro de componentes funcionais. Ele é essencial para que o componente **"lembre" de valores** entre renderizações.

Definição do componente funcional ContadorBasico:

```
export function ContadorBasico() {
```

Figura 3 - Função do Contador
Fonte: Elaborado pelo autor (2025)

Aqui, estamos criando um componente React chamado ContadorBasico, utilizando uma **função** em vez de classe — o que é o padrão na maioria dos projetos modernos com React.

Criação do estado com useState:

```
const [contador, setContador] = useState(0);
```

Figura 4 - Estado inicial do Contador
Fonte: Elaborado pelo autor (2025)

- contador é a **variável de estado** que será exibida na tela.
- setContador é a **função que altera o valor** de contador.
- useState(0) define que o valor inicial de contador será 0.

Dica: sempre que o botão for clicado, chamamos o setContador para **atualizar** o valor, assim, o componente será re-renderizado com o novo valor.

Estrutura do JSX renderizado:

```
  return (
    <div>
      <h2>Contador com useState</h2>
      <p>Valor atual: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>Incrementar</button>
    </div>
  );
}
```

Figura 5 - Estrutura JSX do Contador
Fonte: Elaborado pelo autor (2025)

Explicação linha a linha:

- **<h2>:** apenas um título descritivo da funcionalidade.
- **<p>:** mostra o valor atual do contador, acessando a variável contador.
- **<button onClick={...}>:** botão que escuta o evento de clique e, quando clicado, chama a função **setContador** passando o valor atual + 1.

Resumo do comportamento:

Toda vez que o botão "Incrementar" é clicado:

- O valor de contador é aumentado em 1.
- O React **detecta a mudança de estado**.
- O componente ContadorBasico é **re-renderizado automaticamente** com o novo valor.

Testando o comportamento do useState:

Para testar o comportamento do useState, modifique o arquivo App.jsx do seu projeto React criado com Vite. Em seguida, substitua todo o conteúdo atual pelo código a seguir para criar um contador funcional básico.

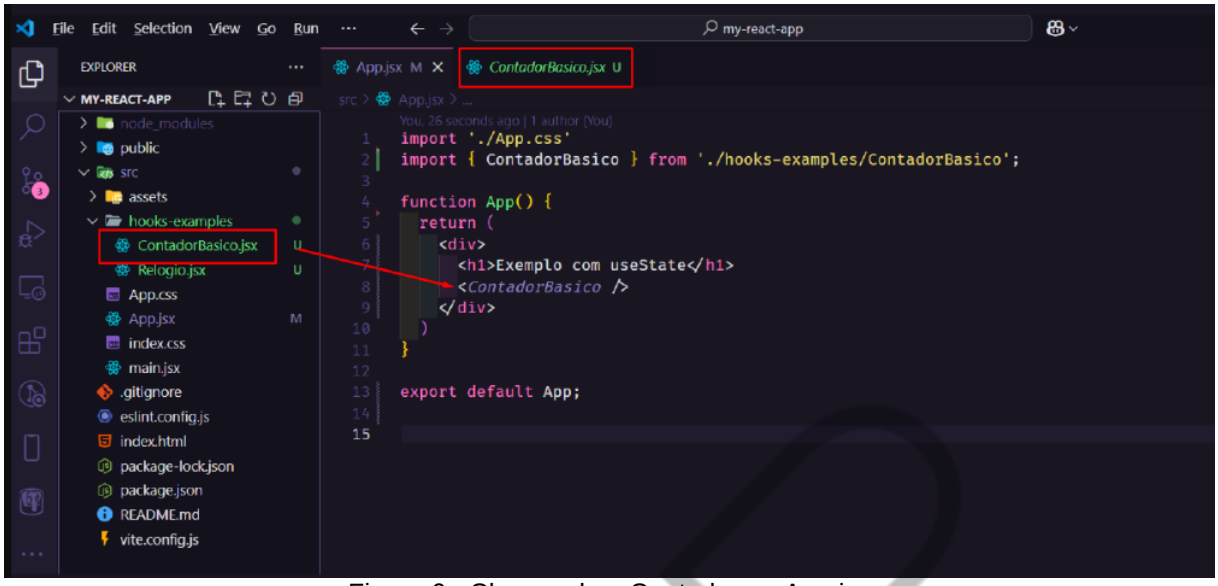


Figura 6 - Chamando o Contador no App.jsx
Fonte: Elaborado pelo autor (2025)

Após modificar o arquivo App.jsx, é necessário **executar o projeto React** para visualizar o comportamento do contador no navegador.

Execute os seguintes passos:

Passo 1 – abra o terminal do VS Code.

- Pressione Ctrl + J no teclado ou acesse o menu superior e clique em "Terminal > Novo Terminal".

Passo 2 – no terminal que abrir na parte inferior, digite o comando:

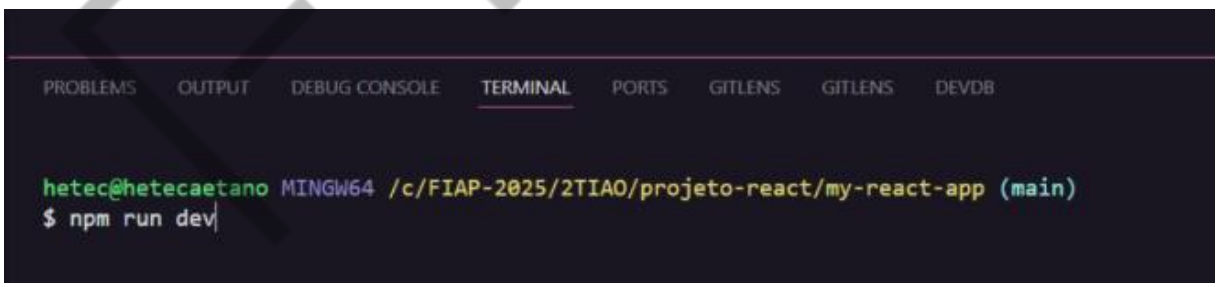


Figura 7 - Executando o projeto
Fonte: Elaborado pelo autor (2025)

O que observar no navegador:

- Acesse: http://localhost:5173.
- Você verá um contador simples iniciado em 0.
- Ao clicar no botão **Incrementar**, o número exibido aumentará.

Isso acontece porque o useState mantém o valor em memória e re-renderiza o componente sempre que ocorrer uma atualização de estado.

2.2 useEffect

O **useEffect** é utilizado para executar efeitos colaterais após a renderização de um componente. Isso inclui chamadas a APIs, *timers*, manipulação do DOM ou integração com serviços externos.

Ele também permite configurar lógicas que devem ser executadas apenas uma vez, sempre que um dado mudar ou ao desmontar o componente.

2.2.1 Exemplo de uso do useEffect – Criando um relógio digital que atualiza sozinho

Criar um arquivo:

Utilize o src/hooks-examples/Relogio.jsx para criar um arquivo, conforme ilustra a figura:

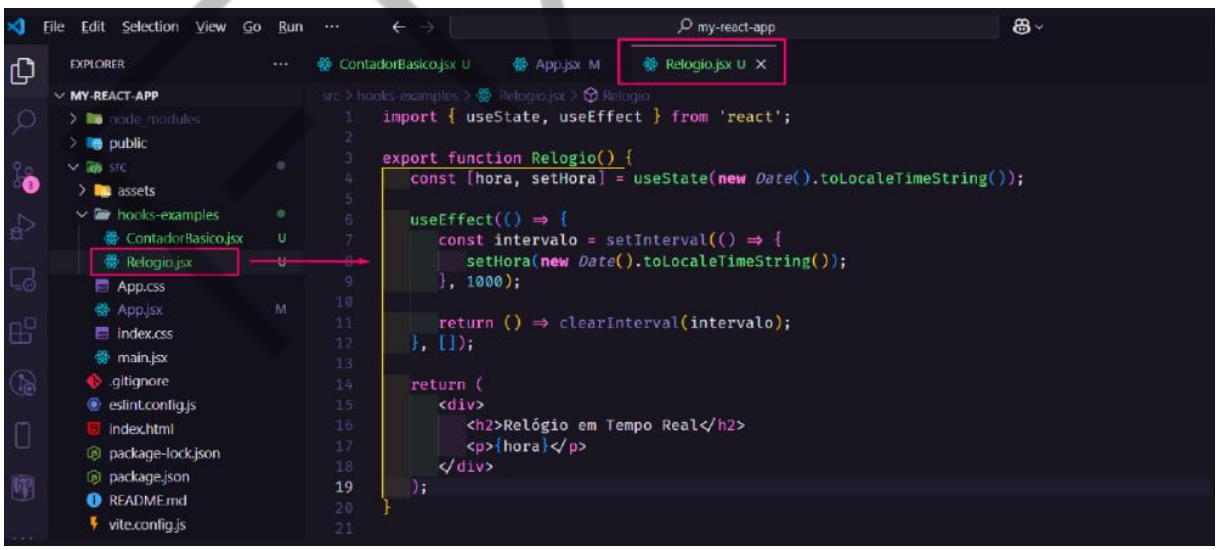


Figura 8 - Exemplo de uso do useEffect
Fonte: Elaborado pelo autor (2025)

Importação dos Hooks necessários:

```
import { useState, useEffect } from 'react';
```

Figura 9 - Importação do hook
Fonte: Elaborado pelo autor (2025)

Neste componente, utilizamos dois Hooks do React:

- **useState:** para armazenar a hora atual.
- **useEffect:** para executar um código repetidamente (a cada segundo) e atualizar o valor da hora automaticamente.

Declaração e inicialização do estado:

```
const [hora, setHora] = useState(new Date().toLocaleTimeString());
```

Figura 10 - Declaração de Estado
Fonte: Elaborado pelo autor (2025)

Criamos um estado chamado hora, que recebe como valor inicial a hora atual formatada como string (HH:MM:SS). Esse valor será atualizado a cada segundo pelo useEffect.

Uso do useEffect para configurar o relógio:

```
useEffect(() => {
    const intervalo = setInterval(() => {
        setHora(new Date().toLocaleTimeString());
    }, 1000);

    return () => clearInterval(intervalo);
}, []);
```

Figura 11 - Uso do useEffect
Fonte: Elaborado pelo autor (2025)

Explicação detalhada por linhas:

- **useEffect(() => { ... }, []):** o efeito será executado **apenas uma vez**, assim que o componente for montado.

- **setInterval(...):** cria um intervalo que executa a função de atualizar a hora a cada **1000** milissegundos (1 segundo).
- **setHora(...):** atualiza o estado com a nova hora.
- **return () => clearInterval(...):** esta função de limpeza é executada automaticamente quando o componente for desmontado, evitando que o intervalo continue rodando em segundo plano. Isso é essencial em React para **evitar o vazamento de memória**.

JSX renderizado na tela:

```
    return (
        <div>
            <h2>Relógio em Tempo Real</h2>
            <p>{hora}</p>
        </div>
    );
}
```

Figura 12 - Render do jsx
Fonte: Elaborado pelo autor (2025)

Explicação: esse trecho é a interface exibida ao usuário:

- **<h2>:** um título explicativo.
- **<p> {hora}:** um parágrafo que exibe dinamicamente o valor de hora, o qual será atualizado automaticamente a cada segundo.

Resumo do funcionamento:

- O useEffect é utilizado aqui para criar um comportamento que se **repete com o passar do tempo** (*timer*).
- A cada segundo, o valor de hora é atualizado com a hora atual.
- Sempre que o estado hora muda, o componente é re-renderizado com o novo valor.

Teste prático – Visualizando o useEffect em ação:

Agora que você entendeu como funciona o Hook useEffect, vamos testar o seu comportamento na prática, executando um componente que exibe a **hora atualizada automaticamente a cada segundo**.

Abra o arquivo App.jsx e **substitua todo o conteúdo** por este:

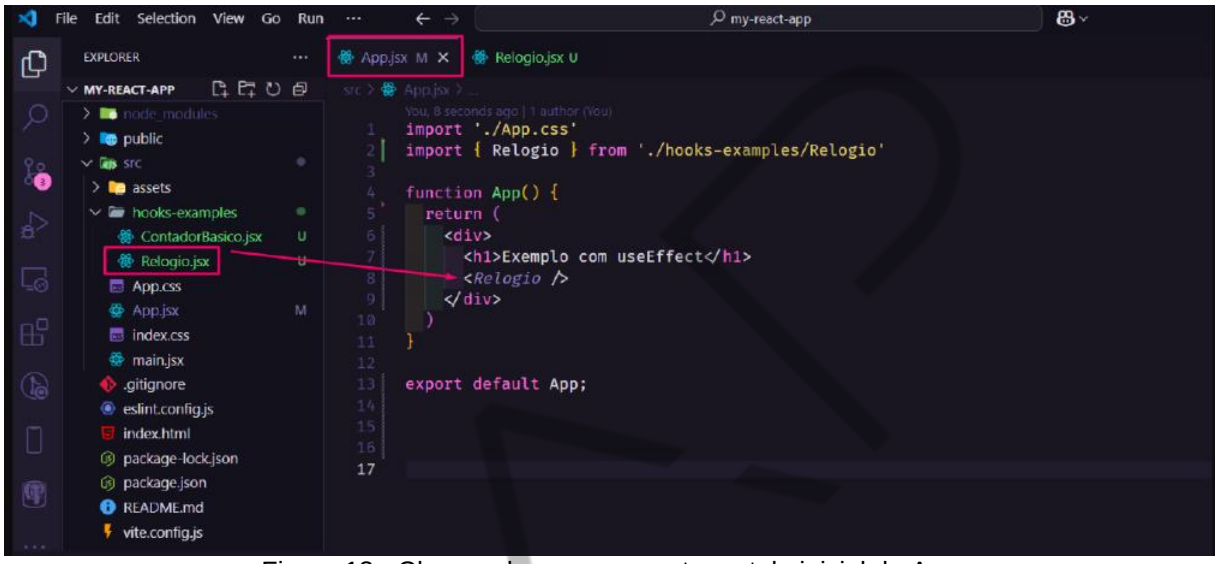


Figura 13 - Chamando o componente na tela inicial do App
Fonte: Elaborado pelo autor (2025)

Atenção: note que a única mudança em relação ao exemplo anterior com useState é que agora estamos chamando o componente **<Relogio />** no lugar de **<ContadorBasico />**.

Isso demonstra uma das boas práticas do React: **a componentização**. Cada exemplo da aula está isolado em seu próprio componente, facilitando a organização do código, a leitura e a manutenção do projeto. Seguiremos esse padrão para os demais exemplos — sempre importando o componente que estamos criando e exibindo no App.jsx.

2.3 useContext

O **useContext** permite acessar valores globais compartilhados entre componentes, eliminando a necessidade de passar props manualmente por toda a hierarquia. É ideal para armazenar dados como tema, autenticação ou preferências

do usuário. Ele funciona em conjunto com a Context API e torna o código mais limpo e fácil de manter em aplicações de médio e grande porte.

2.3.1 Exemplo de uso do useContext – Compartilhando dados entre componentes sem props

Criar uma estrutura de arquivos:

Utilize o src/contexts/TemaContexto.jsx para criar uma estrutura, conforme ilustra a figura:

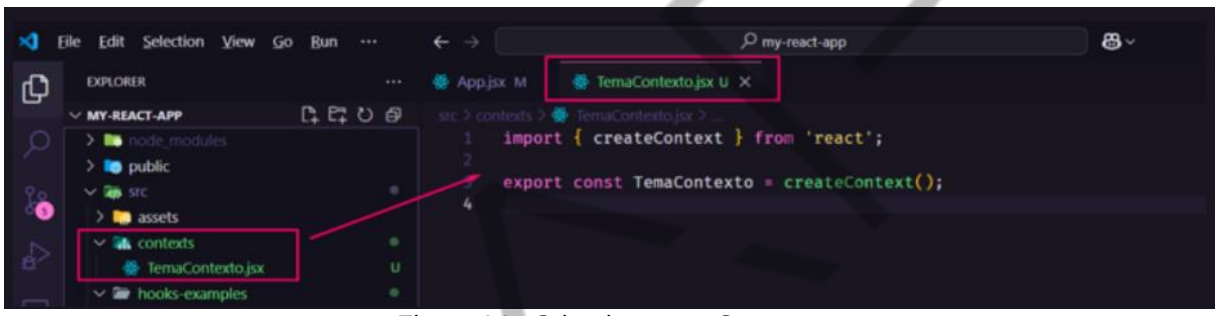


Figura 14 - Criando o temaContexto
Fonte: Elaborado pelo autor (2025)

Explicação:

- createContext() cria um **contexto global**.
- Esse contexto será utilizado para **armazenar e distribuir o valor do tema** (por exemplo: "dark", "light").
- Esse arquivo não tem lógica de exibição — apenas exporta o contexto criado.

Utilize o src/hooks-examples/ComponenteComTema.jsx para criar um componente com tema. Observe:

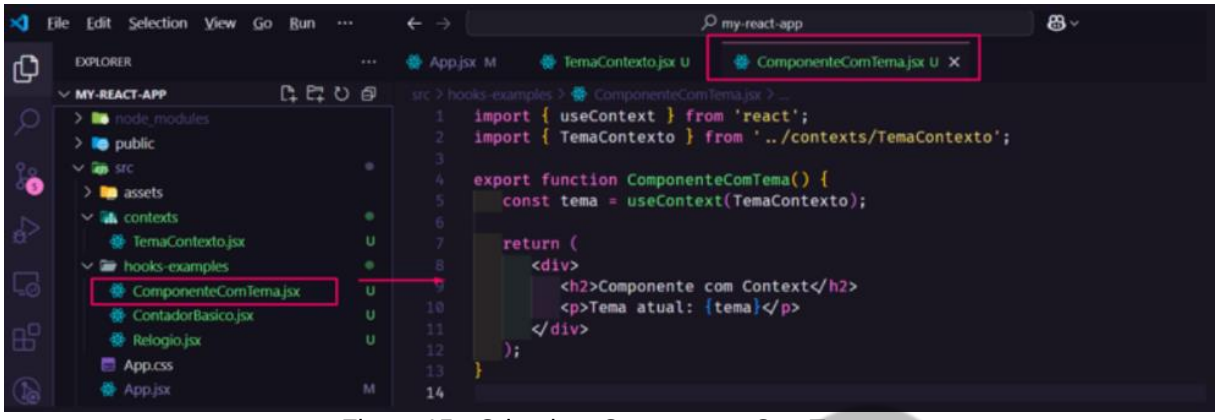


Figura 15 - Criando o ComponenteComTema
Fonte: Elaborado pelo autor (2025)

Explicação por trecho:

- O **useContext(TemaContexto)** acessa o valor atual do contexto.
- O valor retornado (neste caso, "dark") é exibido na tela dentro do parágrafo <p>.
- Esse componente **não precisa receber props** — o contexto já fornece o dado necessário.

Como utilizar no App.jsx:

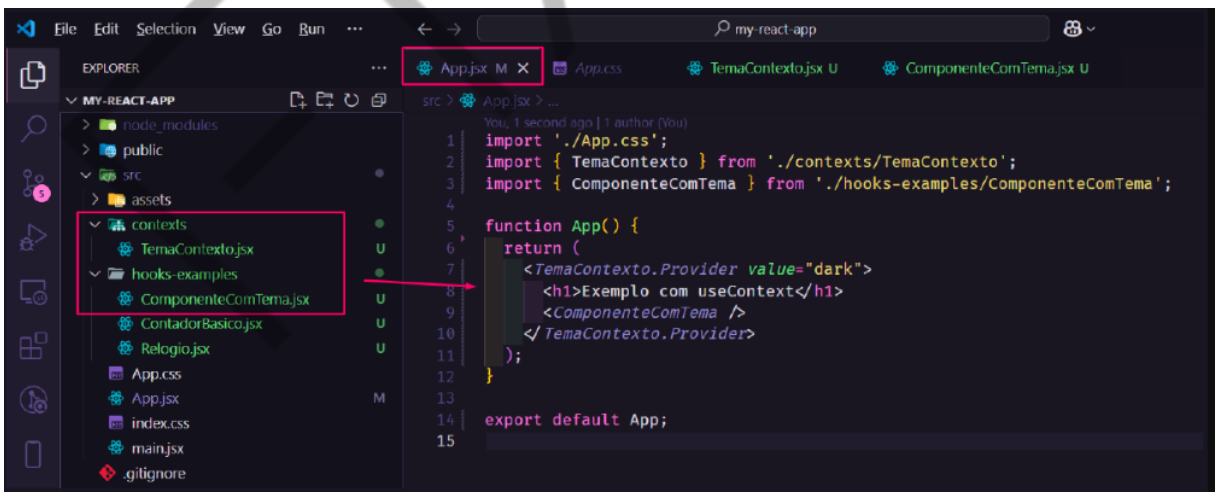


Figura 16 - Chamando o componente na página inicial
Fonte: Elaborado pelo autor (2025)

Dica: para melhorar o exemplo com tema, alterne estilos com CSS.

Na linha 1 do arquivo App.jsx, estamos importando o arquivo de estilo padrão:

import './App.css';

Até agora, esse CSS não foi utilizado nos nossos exemplos. Vamos aproveitar o exemplo do useContext para mostrar na prática como o tema pode afetar o visual da aplicação, alterando a aparência entre modo claro e escuro.

Adicione o seguinte CSS em src/App.css:

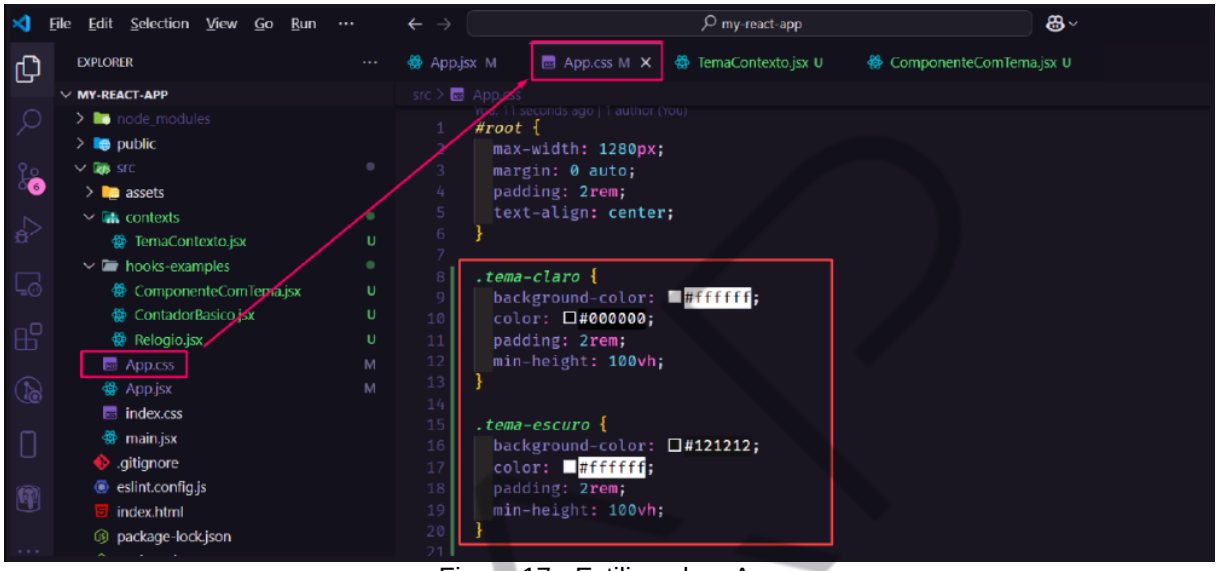


Figura 17 - Estilizando o App
Fonte: Elaborado pelo autor (2025)

Atualize o App.jsx para aplicar a classe dinamicamente:

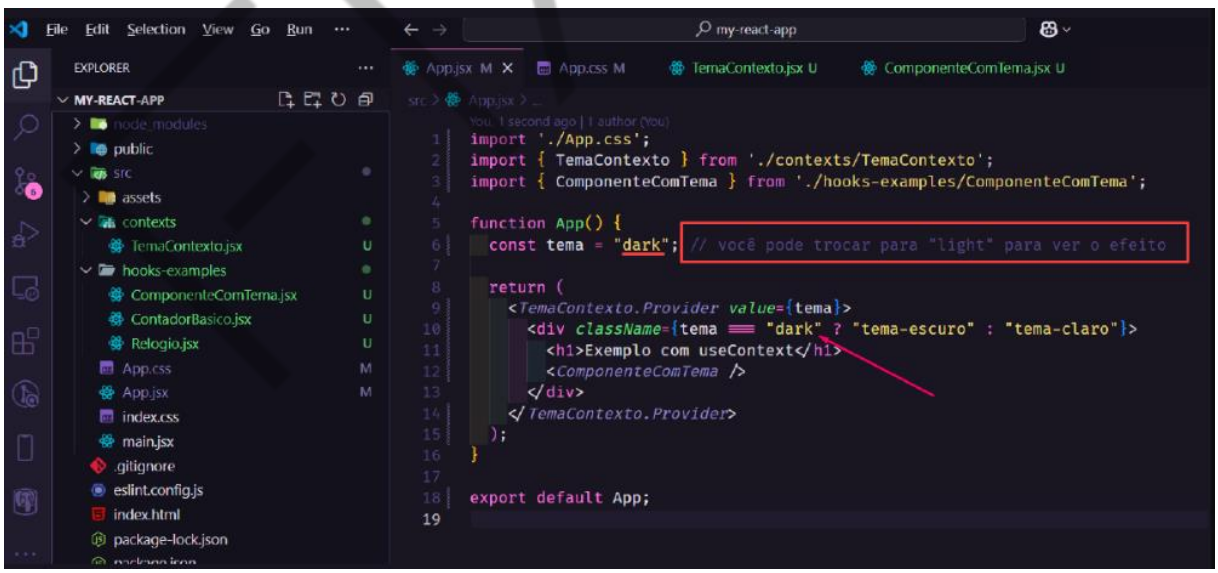


Figura 18 - Chamando o componente na tela inicial
Fonte: Elaborado pelo autor (2025)

O que se deve observar:

- Se tema for "**dark**", o fundo da tela ficará escuro com texto claro.
- Se mudar o valor para "**light**", o fundo será claro com texto escuro.

- Isso mostra como o valor do contexto influencia diretamente o estilo da aplicação.

No trecho anterior, usamos uma estrutura chamada **operador ternário**, que é uma forma abreviada de escrever uma **condicional if/else** diretamente dentro do JSX:

```
<div className={tema === "dark" ? "tema-escuro" : "tema-claro"}>
```

Figura 19 - Switch para troca de tema
Fonte: Elaborado pelo autor (2025)

Como o operador ternário funciona:

A estrutura geral é:

```
condição ? valor_se_verdadeiro : valor_se_falso
```

Figura 20 - Operador ternário para verificação de condição
Fonte: Elaborado pelo autor (2025)

No nosso caso:

- **tema === "dark": condição** que verifica se o tema é escuro.
- **"tema-escuro":** valor retornado **se a condição for verdadeira**.
- **"tema-claro":** valor retornado **se a condição for falsa**.

2.4 useReducer

O **useReducer** é um Hook usado quando precisamos lidar com lógicas de estado mais complexas, como múltiplas ações, condições ou atualizações encadeadas. Ele segue o padrão reducer conhecido do Redux, com estado, dispatch e uma função reducer. É ideal quando o useState se torna difícil de organizar, oferecendo mais previsibilidade e uma melhor separação de responsabilidades.

2.4.1 Exemplo de uso do useReducer – Criando um contador com lógica mais controlada

Criar um arquivo:

Utilize o src/hooks-examples/ContadorReducer.jsx, conforme ilustra a figura:

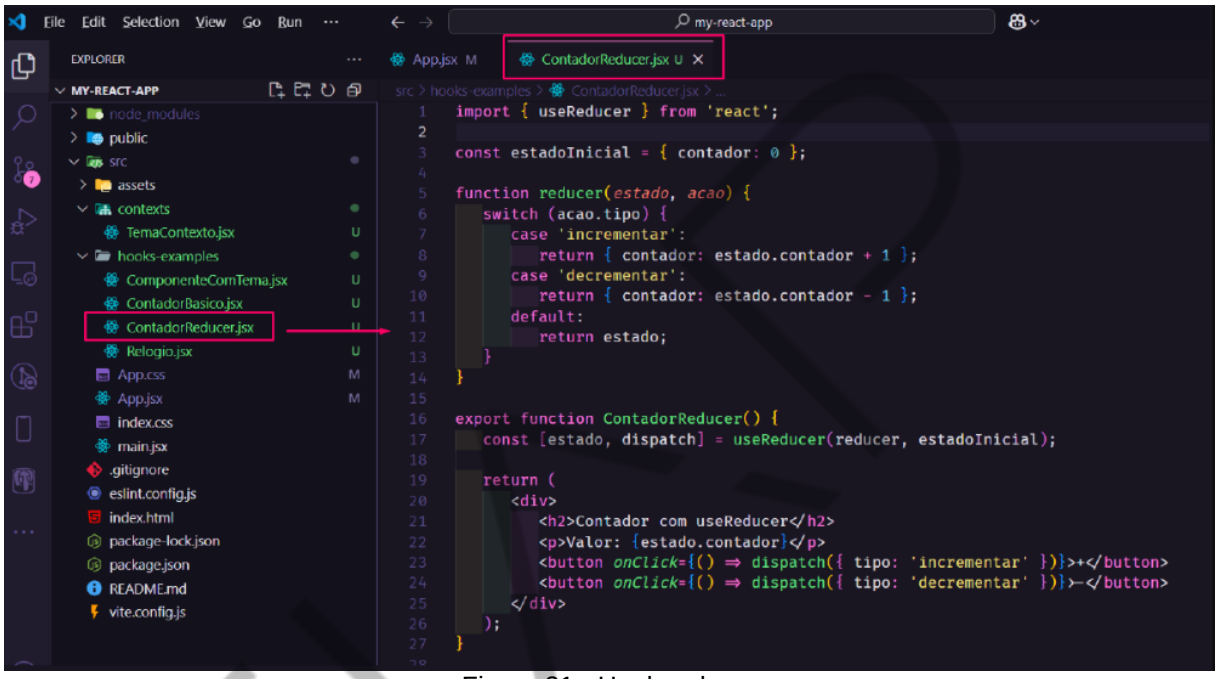


Figura 21 - Hook reducer
Fonte: Elaborado pelo autor (2025)

Importando o Hook e configurando o estado inicial:

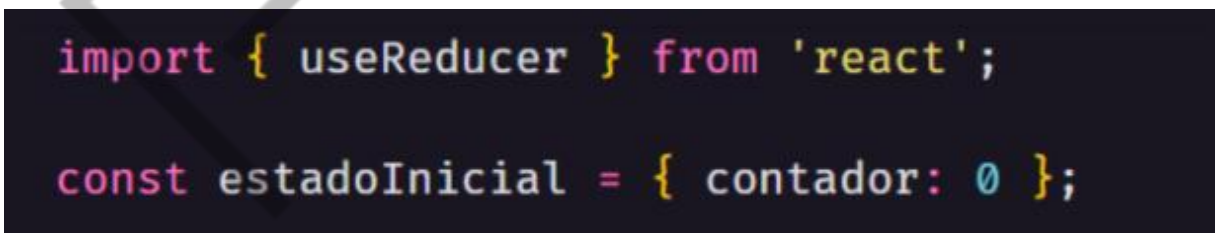


Figura 22 - Hook e estado inicial
Fonte: Elaborado pelo autor (2025)

Explicação:

- Importamos o Hook useReducer do React.
- estadoInicial é um objeto que representa o valor inicial do nosso estado.
- Neste exemplo, o estado contém apenas a propriedade contador, mas poderá incluir outros dados no futuro.

Criando a função reducer:

```
function reducer(estado, acao) {
    switch (acao.tipo) {
        case 'incrementar':
            return { contador: estado.contador + 1 };
        case 'decrementar':
            return { contador: estado.contador - 1 };
        default:
            return estado;
    }
}
```

Figura 23 - Função Reducer
Fonte: Elaborado pelo autor (2025)

Explicação por trecho:

- A função reducer recebe o estado atual e uma ação.
- Com base no tipo da ação, ela retorna um novo estado modificado.
- switch/case é usado para organizar as possíveis ações.
- É nesse ponto que centralizamos todas as regras de atualização do estado.

Criando o componente funcional ContadorReducer:

```
export function ContadorReducer() {
    const [estado, dispatch] = useReducer(reducer, estadoInicial);
```

Figura 24 - Função ContadorReducer
Fonte: Elaborado pelo autor (2025)

Explicação:

- Chamamos useReducer() passando:
  - A função reducer, que contém as regras.
  - O estadoInicial, que define os valores iniciais.
- **estado**: variável que representa o estado atual.
- **dispatch**: função usada para **enviar ações** para o reducer.

JSX com botões que disparam ações:

```jsx
return (
    <div>
        <h2>Contador com useReducer</h2>
        <p>Valor: {estado.contador}</p>
        <button onClick={() => dispatch({ tipo: 'incrementar' })}>+</button>
        <button onClick={() => dispatch({ tipo: 'decrementar' })}>-</button>
    </div>
);
```

Figura 25 - JSX do contador
Fonte: Elaborado pelo autor (2025)

Explicação:

- Mostramos o valor atual de estado.contador na tela.
- O botão + dispara a ação tipo 'incrementar'.
- O botão - dispara a ação tipo 'decrementar'.
- Cada dispatch() chama o reducer, que decide o novo valor do estado com base no tipo da ação.

Testando o componente no App.jsx:

Abra o App.jsx e substitua o conteúdo para visualizar o contador com useReducer:

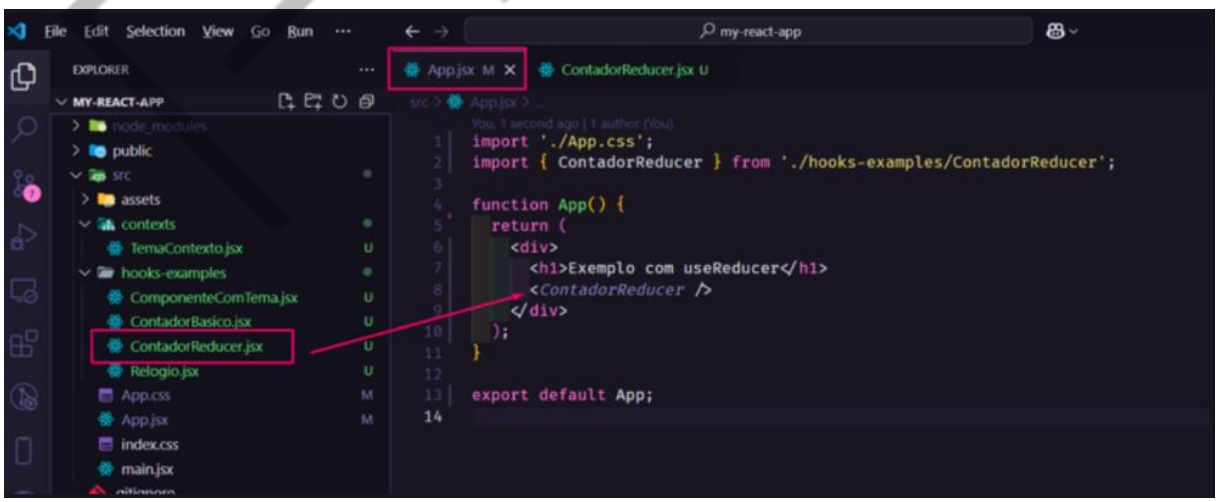


Figura 26 - Chamando o componente Reducer na página inicial
Fonte: Elaborado pelo autor (2025)

Para isso:

- Acesse: <http://localhost:5173>.
- Na tela, você verá dois botões para incrementar e decrementar o valor do contador. Cada clique aciona uma **ação nomeada**, controlada por uma lógica clara e reaproveitável.

FIAP

3 GERENCIAMENTO GLOBAL DE ESTADO COM CONTEXT API

A Context API do React é uma solução leve para **compartilhar dados entre múltiplos componentes** sem precisar passar props manualmente em cada nível. É amplamente utilizada em aplicações para armazenar **temas**, **dados de autenticação**, **idioma**, **status de login**, **carrinhos de compra**, entre outros.

Neste tópico, vamos explorar como criar Providers reutilizáveis, evitar o "*prop drilling*" e conhecer alternativas como Redux e Zustand.

3.1 Criando providers e consumers

3.1.1 Criando o contexto global

Crie uma estrutura de arquivo:

Utilize o src/contexts/ContadorGlobalContext.jsx.

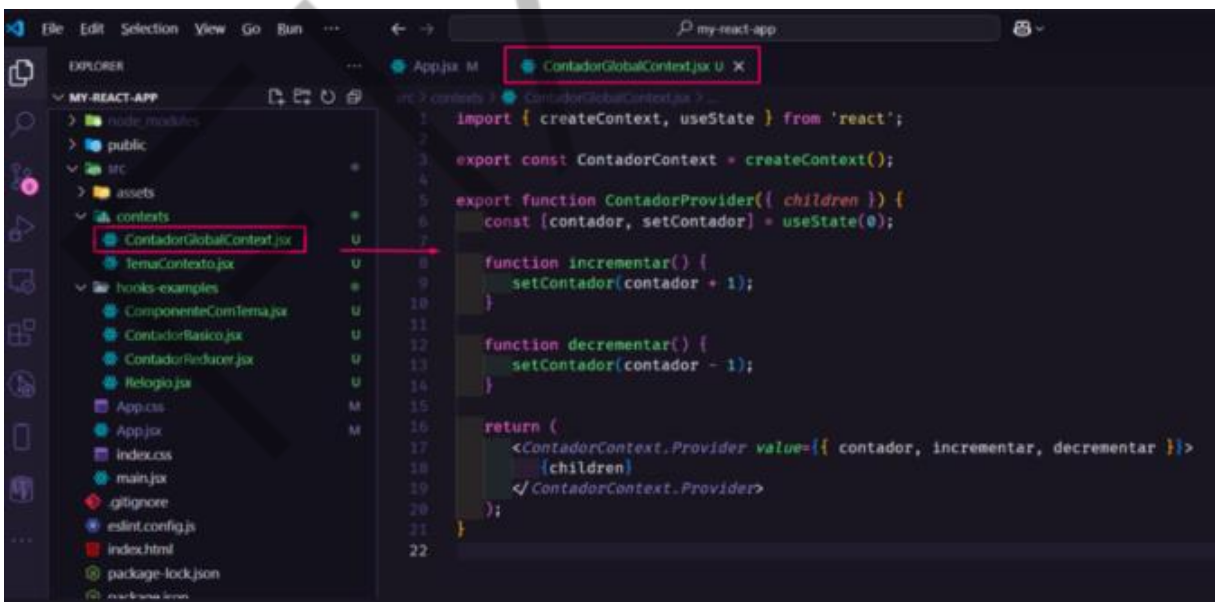


Figura 27 - Criando o contexto global
Fonte: Elaborado pelo autor (2025)

Importação dos Hooks e criação do Contexto:

```
import { createContext, useState } from 'react';

export const ContadorContext = createContext();
```

Figura 28 - Importando os Hooks e criando o contexto
Fonte: Elaborado pelo autor (2025)

Explicação:

- createContext() cria um novo contexto que será utilizado para compartilhar dados globalmente entre componentes.
- ContadorContext é o objeto que usaremos para fornecer e consumir o estado do contador.
- O Hook useState será utilizado dentro do Provider para manter o valor do contador.

Criação do componente Provider:

```
export function ContadorProvider({ children }) {
```

Figura 29 - Componente Provider
Fonte: Elaborado pelo autor (2025)

Explicação:

- Criamos um componente chamado **ContadorProvider**, que envolverá outros componentes e permitirá que eles acessem o contexto.
- A propriedade **children** representa qualquer componente filho que será encapsulado por esse Provider.
- Isso nos permite utilizar o contexto em qualquer ponto da árvore onde esse Provider for aplicado.

Estado local do contador e funções de controle:

```
export function ContadorProvider({ children }) {
  const [contador, setContador] = useState(0);

  function incrementar() {
    setContador(contador + 1);
  }

  function decrementar() {
    setContador(contador - 1);
  }
```

Figura 30 - Estados e Funções do Contador
Fonte: Elaborado pelo autor (2025)

Explicação:

- O **contador** é o valor do contador compartilhado globalmente.
- O **setContador** atualiza esse valor com base nas ações.
- As funções **incrementar()** e **decrementar()** encapsulam a lógica de alteração, facilitando a reutilização em outros componentes.

Retorno do Provider com o estado e ações compartilhados:

```
return (
  <ContadorContext.Provider value={{ contador, incrementar, decrementar }}>
    {children}
  </ContadorContext.Provider>
);
```

Figura 31 - JSX do Contador
Fonte: Elaborado pelo autor (2025)

Explicação:

- O **ContadorProvider** retorna um **ContadorContext**.**Provider**, que torna os dados acessíveis para qualquer componente que estiver dentro do seu escopo.
- A propriedade **value** recebe um objeto com o valor atual (**contador**) e as funções (**incrementar** e **decrementar**).

- Tudo o que estiver dentro de {**children**} terá acesso a esse contexto usando useContext.

3.1.2 Consumindo o contexto em um componente

Crie uma estrutura de arquivo:

Digite src/hooks-examples/ComponenteContadorGlobal.jsx, conforme ilustra a figura:

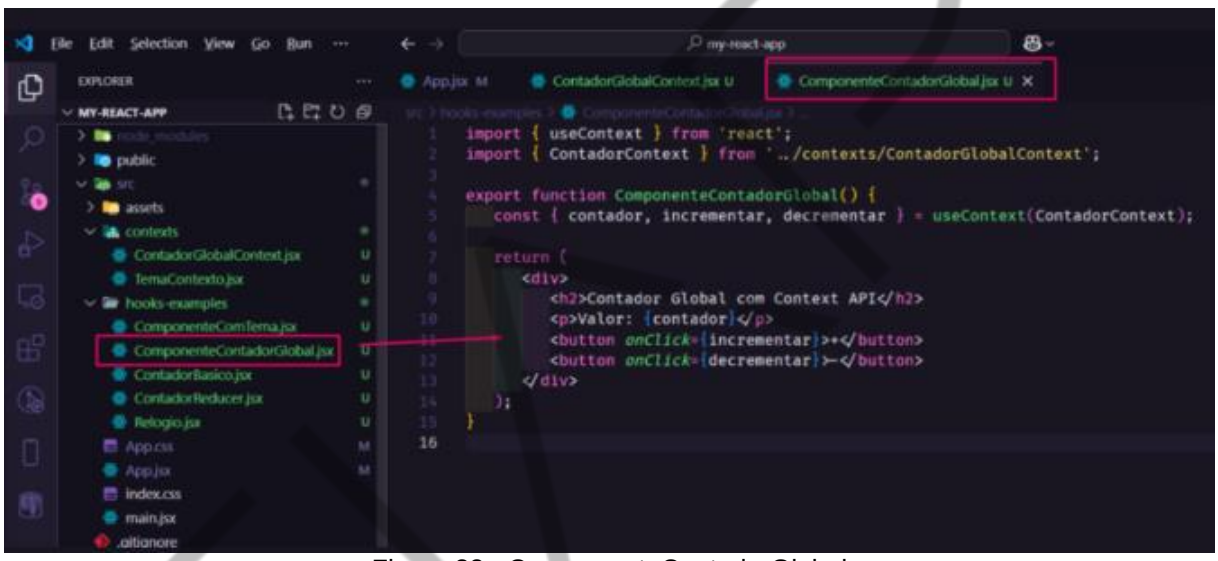


Figura 32 - ComponenteContadorGlobal
Fonte: Elaborado pelo autor (2025)

Explicação por trecho:

- useContext(ContadorContext) acessa o contexto global.
- O componente utiliza o estado e funções fornecidas pelo Provider.
- Não há necessidade de props — o estado está disponível globalmente.

Testando no App.jsx:

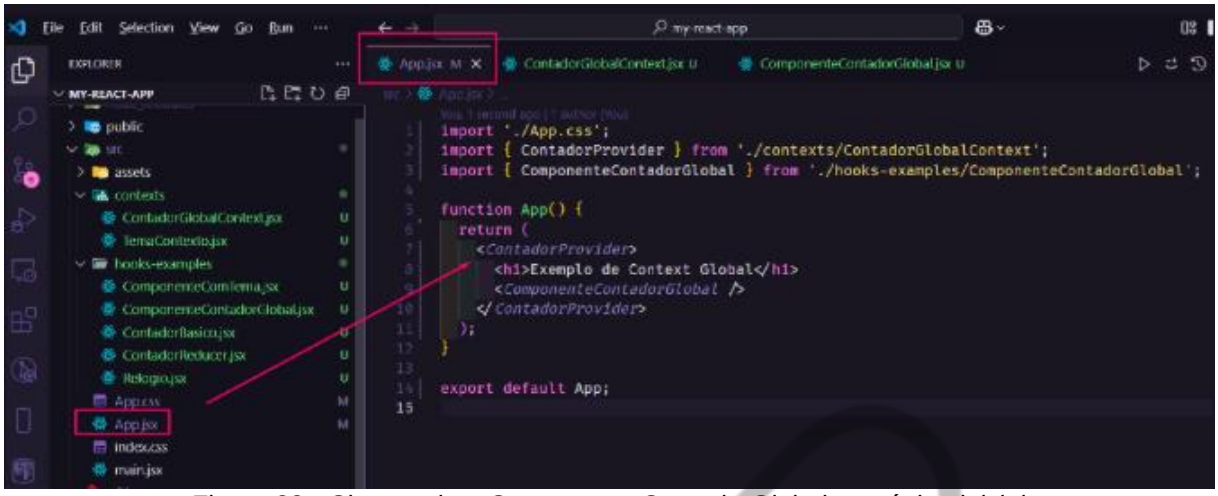


Figura 33 - Chamando o ComponenteContadorGlobal na página inicial
Fonte: Elaborado pelo autor (2025)

O que muda com o uso da Context API:

Nos exemplos anteriores, o estado era controlado localmente dentro de um único componente. Com a Context API, esse estado passa a ser **centralizado em um Provider** e pode ser **acessado por qualquer componente** da aplicação que esteja dentro dele.

Não é mais necessário passar props manualmente. Com o uso do **useContext()**, os componentes acessam os dados de forma direta, tornando o código mais limpo, organizado e reutilizável.

Como funciona na prática:

- O **ContadorProvider** fornece **estado** e **funções** para os componentes envolvidos.
- O **ComponenteContadorGlobal** consome esses dados diretamente, sem precisar receber props.
- Isso permite **reaproveitar a lógica do contador** em diferentes partes do app com facilidade e sem repetição.

3.2 Evitando o “prop drilling” com context

O **Prop Drilling** é o problema de precisar passar props por muitos níveis de componentes intermediários apenas para que um componente lá embaixo os utilize.

A Context API resolve esse problema ao permitir que qualquer componente, **em qualquer nível**, acesse os dados diretamente do contexto, sem depender de passagem manual. Exemplos comuns incluem: autenticação, tema, idioma, notificações.

3.3 Boas práticas e limitações

Boas práticas:

- Separe o contexto em arquivos dentro de /contexts;
- Use Providers apenas em casos necessários, não globalize tudo sem motivo;
- Combine a Context API com outros Hooks (como useReducer) para casos mais complexos.

Limitações:

- **Re-renderizações** em todos os consumidores ao mudar o contexto;
- Não é ideal para **estados altamente mutáveis** ou com **grande volume de dados;**
- A Context API **não substitui** bibliotecas como Redux em todos os casos.

3.4 Alternativas modernas: Redux e zustand (visão geral)

- **Redux:** mais completo, excelente para aplicações complexas com muitos fluxos de dados e controle detalhado sobre o estado. Requer mais configuração, mas oferece plugins e devtools avançadas.
- **Zustand:** é uma biblioteca moderna, baseada em Hooks, com API simples e foco em performance. Permite compartilhar estado com menos re-renderizações e sem o uso de Provider.

4 INTEGRAÇÃO COM APIS DE INTELIGÊNCIA ARTIFICIAL

4.1 O papel das APIs de IA em aplicações modernas

As APIs de Inteligência Artificial se tornaram essenciais em aplicações modernas. Elas oferecem recursos como classificação de texto, análise de imagem, tradução, reconhecimento de voz e geração de conteúdo, sem exigir que o desenvolvedor implemente os algoritmos. Entre as principais fornecedoras estão OpenAI, Google Cloud, AWS, entre outras.

Essas integrações têm ganhado espaço no frontend para:

- Gerar sugestões inteligentes.
- Interpretar comandos de texto ou voz.
- Resumir, traduzir ou classificar textos.
- Criar assistentes virtuais.
- Automatizar respostas e interações.

4.2 Consumo de APIs com fetch e axios

Para integrar APIs de Inteligência Artificial em um projeto React, podemos utilizar fetch (nativo do JavaScript) ou axios, uma biblioteca que simplifica o envio de requisições HTTP e o tratamento de erros.

Vamos demonstrar ambos os métodos em componentes separados.

4.2.1 Exemplo com fetch

Crie o arquivo **src/hooks-examples/FetchExemplo.jsx**:

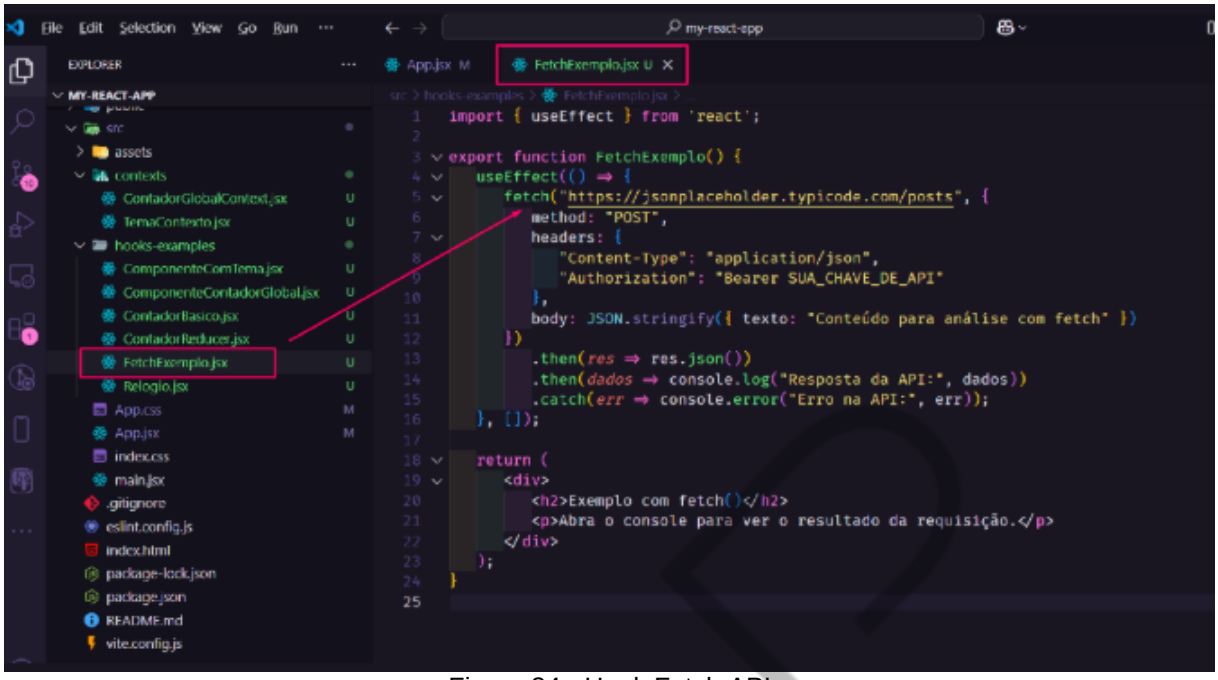


Figura 34 - Hook Fetch API
Fonte: Elaborado pelo autor (2025)

Explicando cada parte do código:

Importação do hook useEffect:

```
import { useEffect } from 'react';
```

Código-fonte 1 - Importação do hook useEffect
Fonte: Elaborado pelo autor (2025)

Este Hook será utilizado para disparar a requisição à API assim que o componente for carregado na tela, como um efeito colateral.

Declaração do componente FetchExemplo:

```
export function FetchExemplo() {
```

Código-fonte 2 - Declaração do componente FetchExemplo
Fonte: Elaborado pelo autor (2025)

É uma função React que será usada na aplicação principal (App.jsx) como um exemplo de consumo de API com fetch

Início do useEffect:

```
useEffect(() => {
```

Código-fonte 3 - Inicialização do useEffect
Fonte: Elaborado pelo autor (2025)

O código aqui dentro será executado uma única vez, assim que o componente for montado na tela (graças ao [] no final).

Requisição HTTP com fetch:

```
          fetch("https://jsonplaceholder.typicode.com/posts", {
               method: "POST",
               headers: {
                  "Content-Type": "application/json",
                  "Authorization": "Bearer SUA_CHAVE_DE_API"
               },
               body:  JSON.stringify({  texto:  "Conteúdo  para
análise com fetch" })
            })
```

Código-fonte 4 - Requisição HTTP com fetch
Fonte: Elaborado pelo autor (2025)

Conversão da resposta em JSON:

```
      .then(res => res.json())
```

Código-fonte 5 - Conversão da resposta em JSON
Fonte: Elaborado pelo autor (2025)

Quando a resposta chega, ela é transformada em um objeto JavaScript para ser manipulada.

Exibição do resultado no console do navegador:

```
      .then(dados => console.log("Resposta da API:", dados))
```

Código-fonte 6 - Exibição de dados no console
Fonte: Elaborado pelo autor (2025)

A resposta da API é exibida no console. Mas o conteúdo ainda não aparece na tela da aplicação.

Tratamento de erro:

```
      .catch(err => console.error("Erro na API:", err));
```

Código-fonte 7 - Exibindo console error
Fonte: Elaborado pelo autor (2025)

Caso a requisição falhe (exemplo: token inválido, servidor offline), será exibida uma mensagem de erro no console.

Dependência vazia do useEffect:

```
    }, []);
```

Código-fonte 8 - Dependência inicializada vazia no useEffect
Fonte: Elaborado pelo autor (2025)

Isso garante que o efeito seja executado apenas uma vez, na montagem do componente

JSX que será renderizado:

```
        return (
            <div>
                <h2>Exemplo com fetch()</h2>
                <p>Abra o console para ver o resultado da
requisição.</p>
            </div>
        );
    }
```

Código-fonte 9 - Render do JSX
Fonte: Elaborado pelo autor (2025)

Exibe um título e uma orientação para que o usuário abra o console (F12) e veja a resposta retornada pela API.

Como testar no App.jsx:

Abra o arquivo src/App.jsx e substitua o conteúdo por:

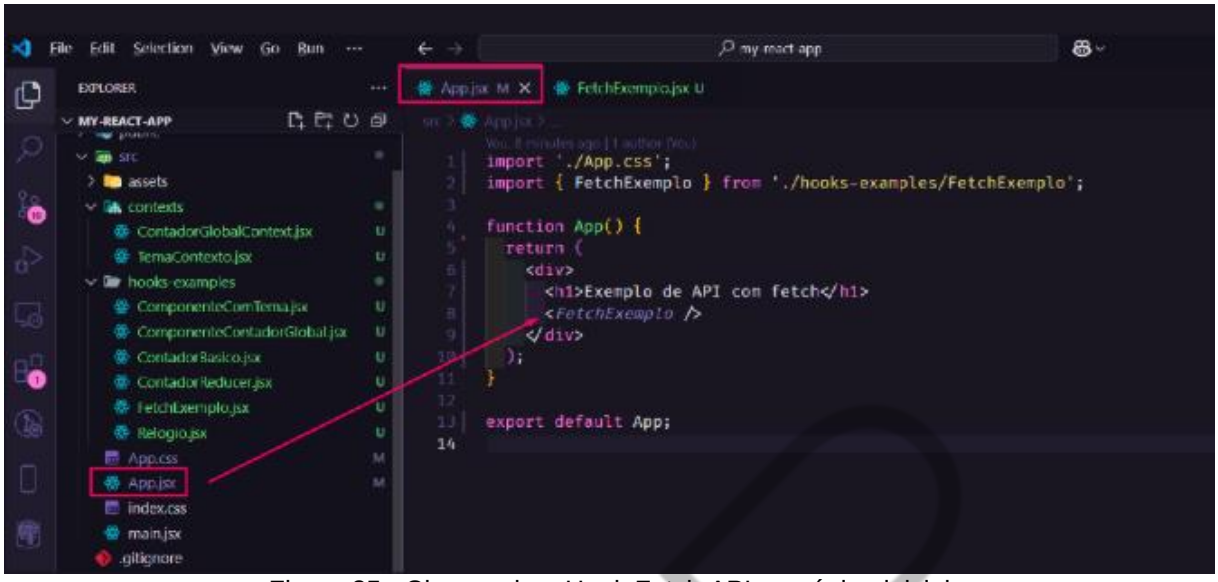


Figura 35 - Chamando o Hook Fetch API na página inicial
Fonte: Elaborado pelo autor (2025)

Como executar o projeto:

- **Passo 1:** abra o terminal no VS Code com Ctrl + J.
- **Passo 2:** veja se o projeto ainda está rondando conforme os exemplos anteriores. Se caso não esteja, digite: **npm run dev**.
- **Passo 3:** no navegador, acesse <http://localhost:5173>.
- **Passo 4:** pressione F12 e abra a aba Console para ver o retorno da API.

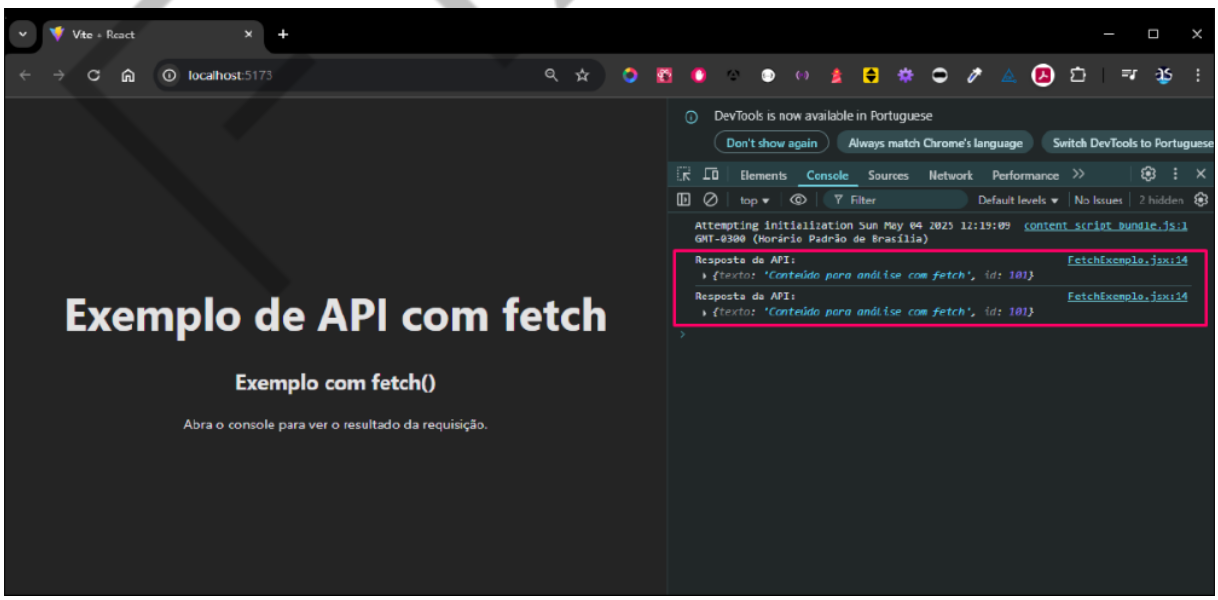


Figura 36 - Visualização no browser do Hook Fetch API
Fonte: Elaborado pelo autor (2025)

4.2.2 Exemplo com axios

O **axios** é uma biblioteca JavaScript que facilita a realização de requisições **HTTP**, como **GET**, **POST**, **PUT** e **DELETE**. Ele é muito utilizado no desenvolvimento frontend por possuir uma sintaxe mais simples e recursos avançados para tratar erros, cabeçalhos e autenticação.

Diferente do **fetch**, que já é nativo do navegador, o **axios** precisa ser instalado manualmente no projeto.

Para isso, siga os passos:

- **Passo 1:** abra o terminal no VS Code (atalho Ctrl + J);
- **Passo 2:** digite o comando abaixo na raiz do seu projeto React: **npm install axios**.

Esse comando irá:

- Baixar a biblioteca axios e salvá-la na pasta node_modules;
- Atualizar automaticamente o package.json com a dependência;
- Permitir que seja possível importar o axios nos componentes.

Após a instalação, já podemos usar o axios normalmente no projeto.

Agora, crie um arquivo **src/hooks-examples/AxiosExemplo.jsx**:

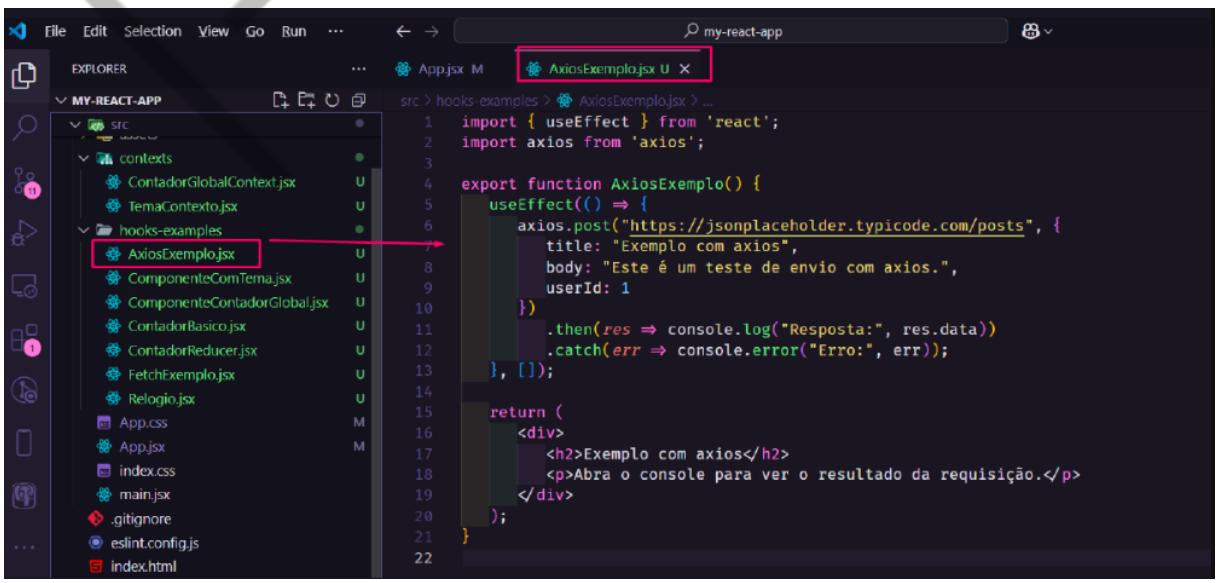


Figura 37 - Hook Axios API
Fonte: Elaborado pelo autor (2025)

Explicando cada parte do código:

Importação de recursos:

```
import { useEffect } from 'react';
import axios from 'axios';
```

Código-fonte 10 - Importação de recursos do React
Fonte: Elaborado pelo autor (2025)

- **useEffect:** permite disparar uma ação assim que o componente for carregado.
- **axios:** biblioteca que facilita requisições HTTP, com sintaxe mais limpa que fetch, especialmente para métodos POST e cabeçalhos.

Declaração do componente AxiosExemplo:

```
export function AxiosExemplo() {
```

Código-fonte 11 - Declaração do componente AxiosExemplo
Fonte: Elaborado pelo autor (2025)

Este componente será utilizado no App.jsx para demonstrar uma chamada real a uma API de teste com o axios.

Início do efeito colateral (useEffect):

```
useEffect(() => {
```

Código-fonte 12 - Início de efeitos com o hook useEffect
Fonte: Elaborado pelo autor (2025)

A função dentro do useEffect será executada **assim que o componente for montado na tela**, uma única vez (graças ao array de dependência vazio).

Requisição POST com axios:

```
axios.post("https://jsonplaceholder.typicode.com/posts", {
          title: "Exemplo com axios",
          body: "Este é um teste de envio com axios.",
          userId: 1
        })
```

Código-fonte 13 - Uso do axios para requisição POST
Fonte: Elaborado pelo autor (2025)

- URL: jsonplaceholder.typicode.com/posts é uma API pública para simular requisições.
- O corpo da requisição envia um objeto com título, corpo e um ID de usuário fictício.
- O axios.post() já transforma o objeto automaticamente em JSON e envia com os cabeçalhos corretos.

Tratamento da resposta:

```
            .then(res => console.log("Resposta:", res.data))
```

Código-fonte 14 - Tratamento de resposta no console log
Fonte: Elaborado pelo autor (2025)

Quando a requisição for concluída com sucesso, os dados retornados são exibidos no console (**res.data**).

Tratamento de erro:

```
            .catch(err => console.error("Erro:", err));
    }, []);
```

Código-fonte 15 - Tratamento de erros com console error
Fonte: Elaborado pelo autor (2025)

Se algo der errado na requisição (como falta de conexão), o erro será exibido no console para depuração.

JSX renderizado na tela:

```
    return (
        <div>
            <h2>Exemplo com axios</h2>
            <p>Abra o console para ver o resultado da
requisição.</p>
        </div>
    );
}
```

Código-fonte 16 - Render JSX da API com axios
Fonte: Elaborado pelo autor (2025)

Mostra um título explicativo para o usuário. A resposta da API não aparece diretamente na tela, e sim no console.

Como testar o componente:

No src/App.jsx, substitua o conteúdo por:

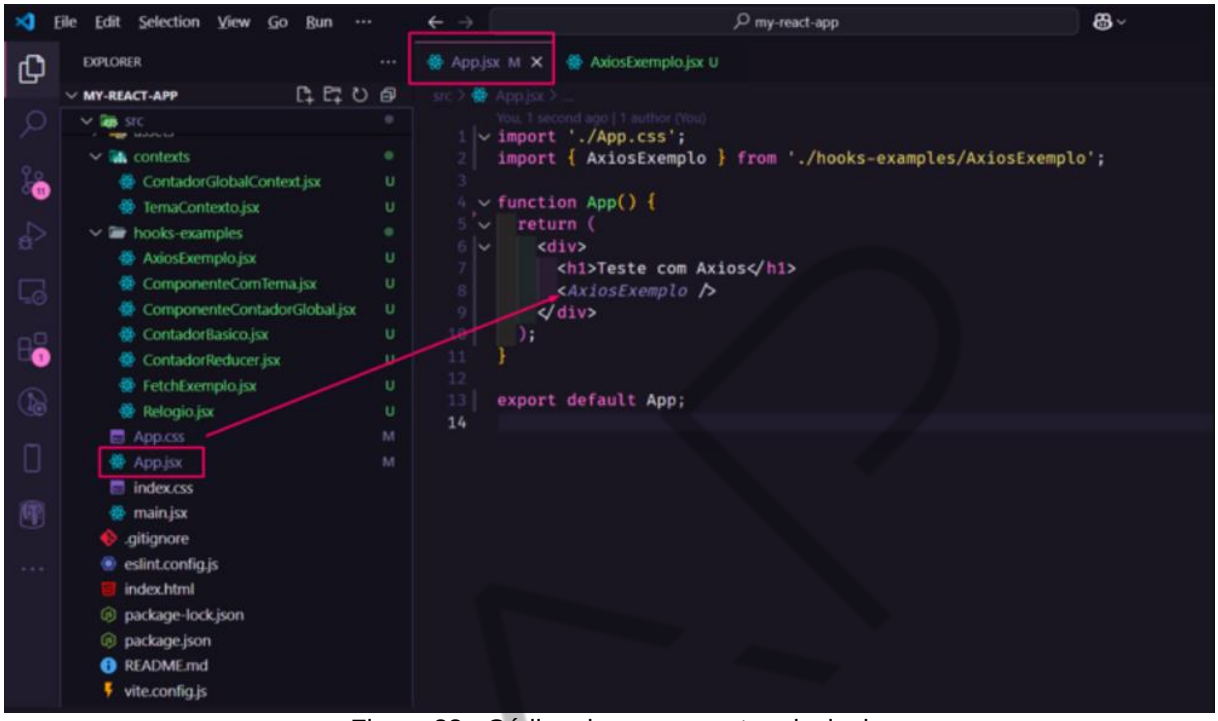


Figura 38 - Código do componente principal
Fonte: Elaborado pelo autor (2025)

Observação: pode rodar da mesma forma da anterior.

Veja o retorno na aba **Console**:

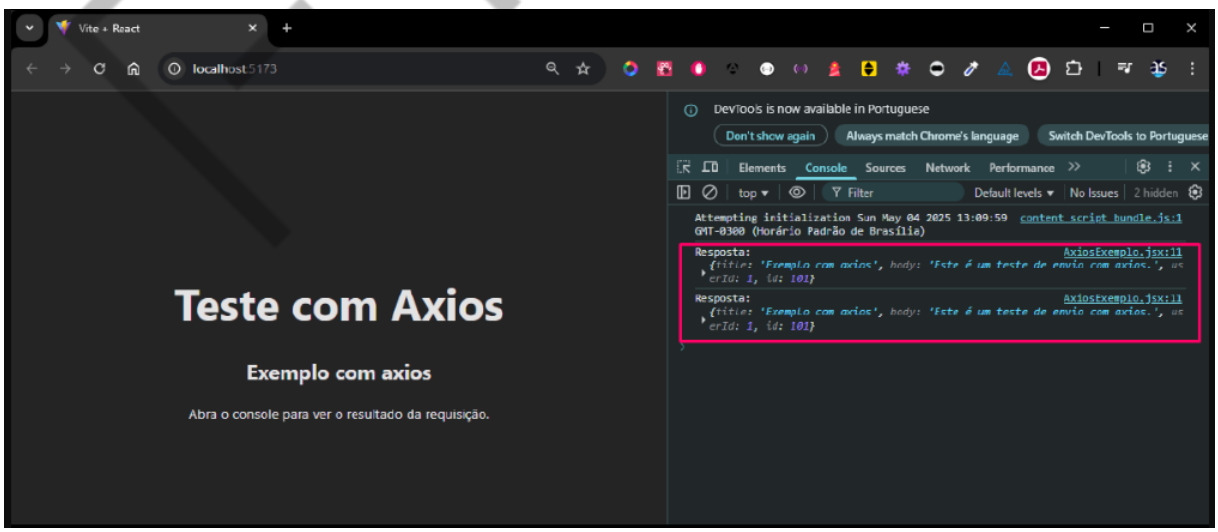


Figura 39 - Visualização no browser do Hook Axios API
Fonte: Elaborado pelo autor (2025)

Importante:
Para testar as requisições no frontend de forma segura e sem a necessidade de autenticação, utilizamos o endpoint público <https://jsonplaceholder.typicode.com/posts>.

Esse serviço simula uma API REST real e retorna dados fictícios em formato JSON. Embora os dados não sejam realmente salvos no servidor, ele responde como se a operação tivesse sido concluída com sucesso, o que é ideal para testes de chamadas fetch e axios no React.

4.3 Gerenciamento de estados de carregamento e erro

Antes de consumirmos **APIs** reais de IA, é importante entender como exibir mensagens de carregamento, erro e resposta.

Vamos criar um exemplo utilizando a API pública jsonplaceholder.typicode.com/posts.

Crie um arquivo **src/hooks-examples/EstadoComFeedback.jsx**:

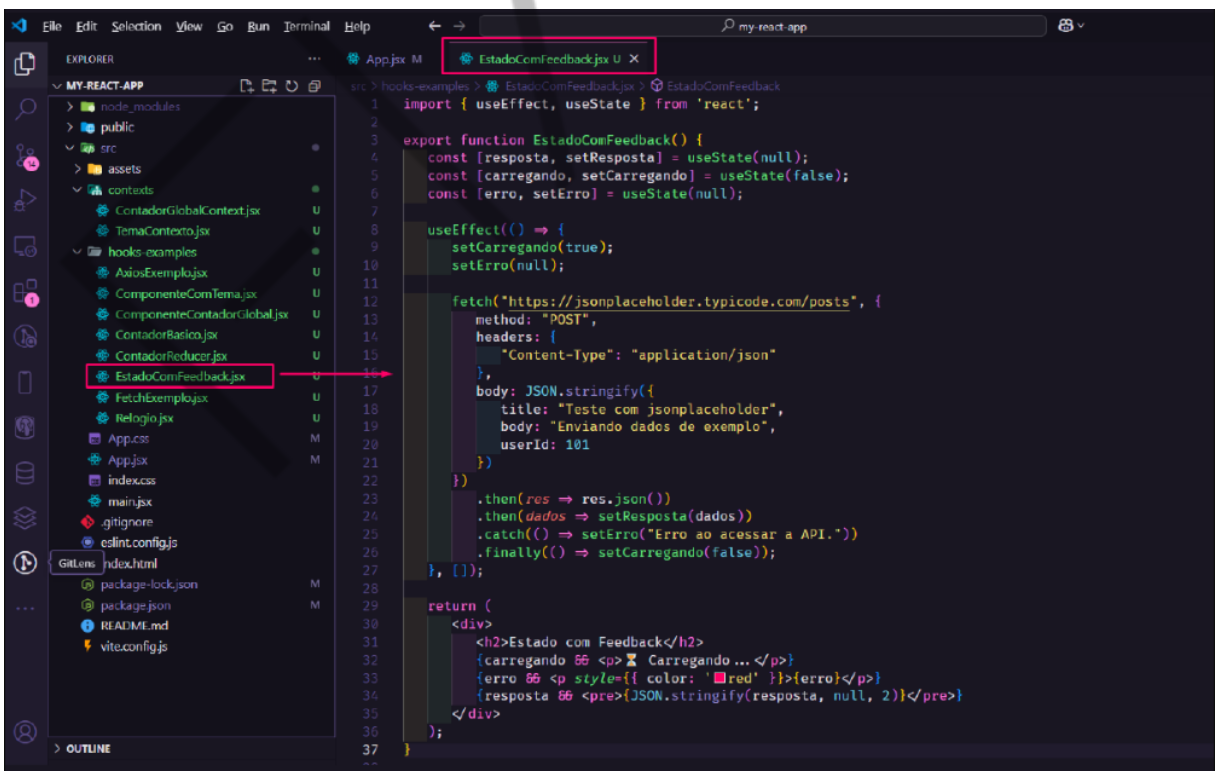


Figura 40 - Gerenciamento de Estado
Fonte: Elaborado pelo autor (2025)

Explicando cada parte do código:

Importação de hooks:

```
import { useEffect, useState } from 'react';
```

Código-fonte 17 - Importação dos hooks useEffect e useState
Fonte: Elaborado pelo autor (2025)

Estamos importando dois hooks fundamentais:

- **useState:** para controlar os dados, estados de carregamento e erro.
- **useEffect:** para realizar a chamada à API assim que o componente for montado.

Declaração do componente:

```
export function EstadoComFeedback() {
```

Código-fonte 18 - Declaração da função do componente
Fonte: Elaborado pelo autor (2025)

Define um componente funcional que será exportado para ser utilizado no App.jsx.

Estados locais:

```
const [resposta, setResposta] = useState(null);
const [carregando, setCarregando] = useState(false);
const [erro, setErro] = useState(null);
```

Código-fonte 19 - Constantes de estados locais
Fonte: Elaborado pelo autor (2025)

- **Resposta:** guarda os dados retornados pela API.
- **Carregando:** controla o estado de carregamento da requisição.
- **Erro:** armazena uma mensagem de erro caso a requisição falhe.

useEffect – chamada à API:

```
useEffect(() => {
  setCarregando(true);
  setErro(null);
```

Código-fonte 20 - Carregando dados da API
Fonte: Elaborado pelo autor (2025)

Assim que o componente for exibido na tela, ele:

- Marca carregando como true.
- Limpa o estado anterior de erro (se houver).

Requisição com fetch:

```
fetch("https://jsonplaceholder.typicode.com/posts", {
   method: "POST",
   headers: {
      "Content-Type": "application/json"
   },
   body: JSON.stringify({
      title: "Teste com jsonplaceholder",
      body: "Enviando dados de exemplo",
      userId: 101
   })
})
```

Código-fonte 21 - Requisição com fetch API
Fonte: Elaborado pelo autor (2025)

Estamos simulando o envio de um novo post para a API fictícia jsonplaceholder.

- O método HTTP usado é POST.
- Os dados do body são convertidos para JSON.

Tratamento da resposta:

```
.then(res => res.json())
.then(dados => setResposta(dados))
.catch(() => setErro("Erro ao acessar a API."))
.finally(() => setCarregando(false));
```

Código-fonte 22 - Feedback de requisição
Fonte: Elaborado pelo autor (2025)

- **.then(res => res.json()):** converte a resposta para JSON.
- **.then(dados => setResposta(dados)):** salva o resultado no estado.
- **.catch(...):** define uma mensagem de erro se algo falhar.
- **.finally(...):** marca carregando como false, independentemente do sucesso ou erro.

Renderização condicional com feedback:

```
      return (
         <div>
            <h2>Estado com Feedback</h2>
            {carregando && <p>⏳ Carregando...</p>}
            {erro && <p style={{ color: 'red' }}>{erro}</p>}
            {resposta  &&  <pre>{JSON.stringify(resposta,  null,
2)}</pre>}
         </div>
      );
```

Código-fonte 23 - Render da requisição com feedback ao usuário
Fonte: Elaborado pelo autor (2025)

- Enquanto carregando for true, mostra “Carregando...”.
- Se erro estiver preenchido, mostra a mensagem em vermelho.
- Se resposta estiver disponível, exibe os dados formatados.

Vamos testar no App.jsx:

No App.jsx, troque o componente atual por:

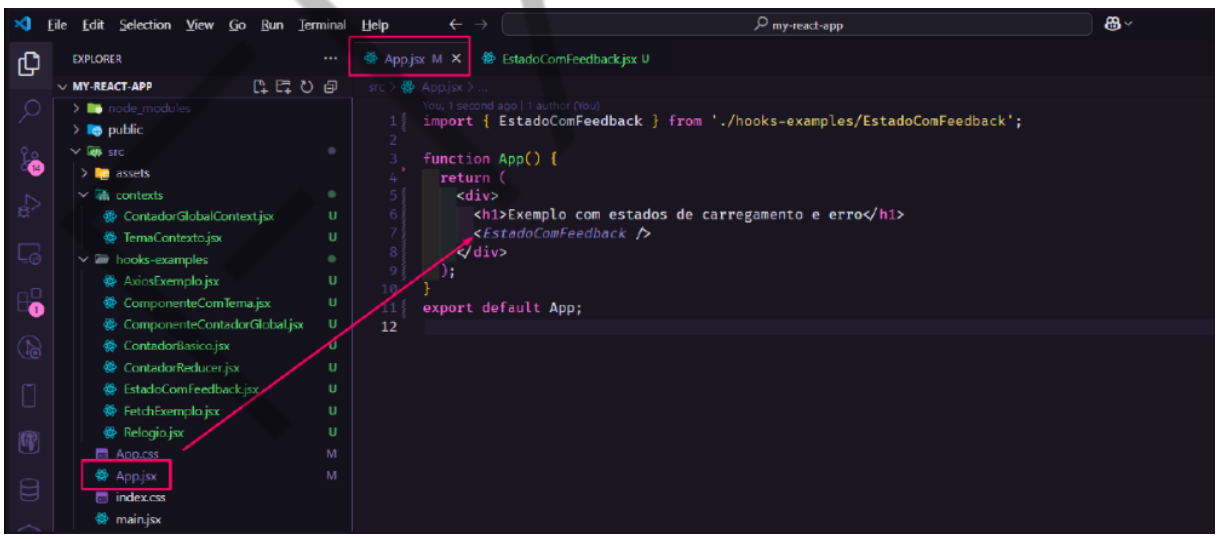


Figura 41 - Chamando o Gerenciamento de Estado na página inicial
Fonte: Elaborado pelo autor (2025)

No browser, veja o **resultado:**

```
    title, body e userId: São os dados que enviamos no body da
requisição                    com                    fetch.
Esses     campos     foram     definidos     no     trecho:
body: JSON.stringify({
                title: "Teste com jsonplaceholder",
                body: "Enviando dados de exemplo",
                userId: 101
            })
```

Código-fonte 24 - Retorno visual ao usuário no browser
Fonte: Elaborado pelo autor (2025)

Id é um campo gerado automaticamente pela API fictícia jsonplaceholder. Mesmo não havendo um banco de dados real por trás, essa API simula a criação de um novo recurso atribuindo um id incremental (neste caso, 101).

4.4 Exemplo prático: Gerador de respostas com IA

Neste exemplo, vamos utilizar uma API real de Inteligência Artificial: a **API da OpenAI**, que permite gerar textos com base em perguntas enviadas pelos usuários.

Vamos simular um pequeno chatbot inteligente, no qual o usuário digita uma pergunta e recebe uma resposta automática gerada pela IA.

Esse tipo de integração é bastante comum em interfaces conversacionais, assistentes virtuais e áreas que precisam de análise de linguagem natural (NLP).

Crie um arquivo em **src/hooks-examples/GeradorRespostaIA.jsx**:

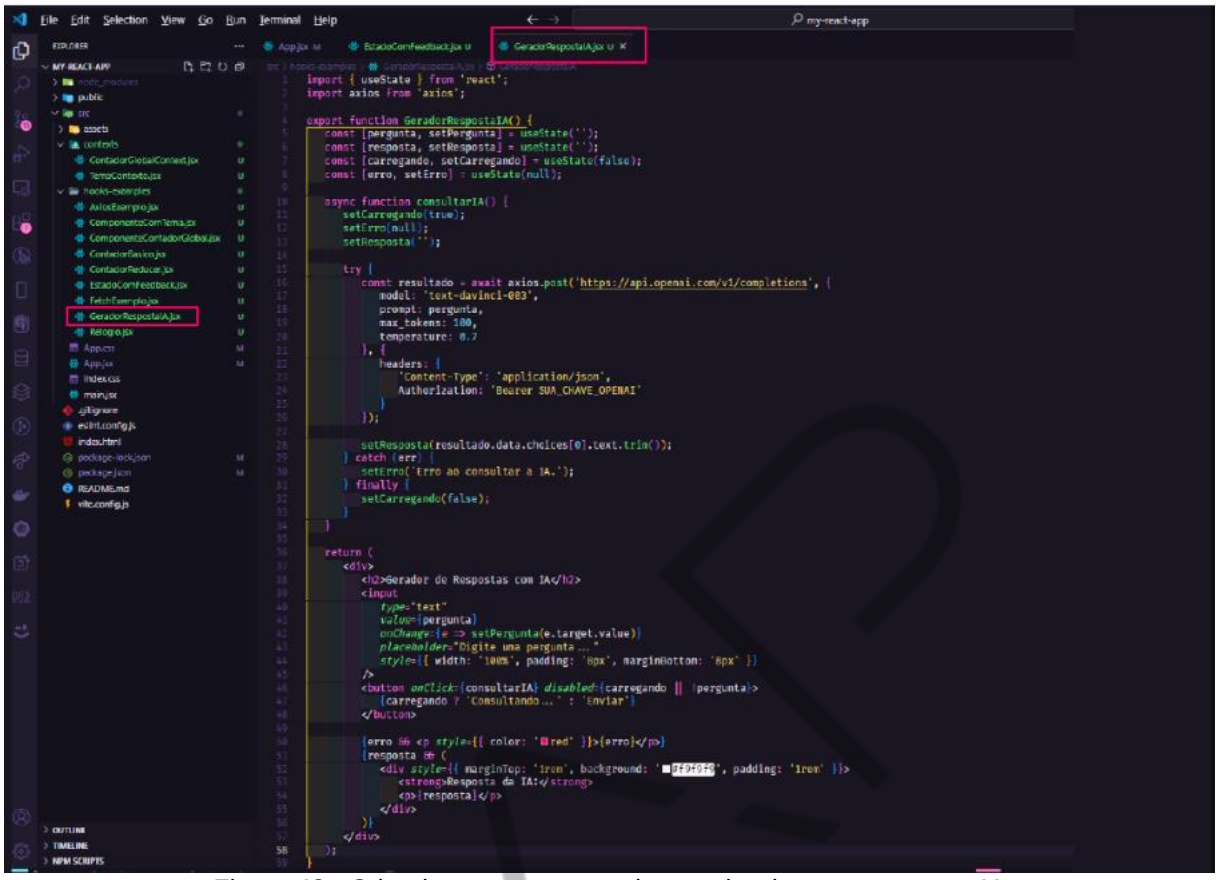


Figura 42 - Criando o componente de gerador de respostas com IA
Fonte: Elaborado pelo autor (2025)

Cópia do conteúdo digitado do código:

```
    import { useState } from 'react';
    import axios from 'axios';

    export function GeradorRespostaIA() {
       const [pergunta, setPergunta] = useState('');
       const [resposta, setResposta] = useState('');
       const [carregando, setCarregando] = useState(false);
       const [erro, setErro] = useState(null);

       async function consultarIA() {
          setCarregando(true);
          setErro(null);
          setResposta('');

          try {
                              const     resultado     =     await
axios.post('https://api.openai.com/v1/completions', {
                model: 'text-davinci-003',
                prompt: pergunta,
                max_tokens: 100,
                temperature: 0.7
             }, {
```

```
                headers: {
                   'Content-Type': 'application/json',
                   Authorization: 'Bearer SUA_CHAVE_OPENAI'
                }
             });

             setResposta(resultado.data.choices[0].text.trim()
);
          } catch (err) {
             setErro('Erro ao consultar a IA.');
          } finally {
             setCarregando(false);
          }
       }

       return (
          <div>
             <h2>Gerador de Respostas com IA</h2>
             <input
                type="text"
                value={pergunta}
                onChange={e => setPergunta(e.target.value)}
                placeholder="Digite uma pergunta..."
                    style={{ width: '100%', padding: '8px',
marginBottom: '8px' }}
             />
            <button onClick={consultarIA} disabled={carregando
|| !pergunta}>
                {carregando ? 'Consultando...' : 'Enviar'}
             </button>

             {erro && <p style={{ color: 'red' }}>{erro}</p>}
             {resposta && (
                 <div style={{ marginTop: '1rem', background:
'#f9f9f9', padding: '1rem' }}>
                   <strong>Resposta da IA:</strong>
                   <p>{resposta}</p>
                </div>
             )}
          </div>
       );
    }
```

Código-fonte 25 - Código fonte da imagem para testes
Fonte: Elaborado pelo autor (2025)

A seguir, veremos as explicações por trechos.

Declaração de estados:

```
      const [pergunta, setPergunta] = useState('');
      const [resposta, setResposta] = useState('');
      const [carregando, setCarregando] = useState(false);
      const [erro, setErro] = useState(null);
```

Código-fonte 26 - Declaração de estados usando hooks
Fonte: Elaborado pelo autor (2025)

Aqui, controlamos:

- O texto da pergunta enviada pelo usuário.
- A resposta recebida da IA.
- O estado de carregamento durante a requisição.
- Possíveis erros.

Função consultarIA():

```
      const resultado = await
axios.post('https://api.openai.com/v1/completions', {
        model: 'text-davinci-003',
        prompt: pergunta,
        max_tokens: 100,
        temperature: 0.7
      }, {
        headers: {
          Authorization: 'Bearer SUA_CHAVE_OPENAI'
        }
      });
```

Código-fonte 27 - Resposta assíncrona de requisição com axios
Fonte: Elaborado pelo autor (2025)

- Envia a pergunta para o modelo da OpenAI (text-davinci-003).
- Retorna um texto gerado com base no prompt.
- O campo max_tokens define o limite de palavras da resposta.
- O temperature define a criatividade da resposta (de 0 a 1).

Exibição dos resultados:

```
{erro && <p style={{ color: 'red' }}>{erro}</p>}
{resposta && (
  <div>
    <strong>Resposta da IA:</strong>
    <p>{resposta}</p>
  </div>
)}
```

Código-fonte 28 - Exibição de erro na tela
Fonte: Elaborado pelo autor (2025)

Mostra feedback de erro (se houver) ou a resposta gerada pela IA.

Teste no App.jsx:

Abra o arquivo src/App.jsx e substitua o conteúdo.

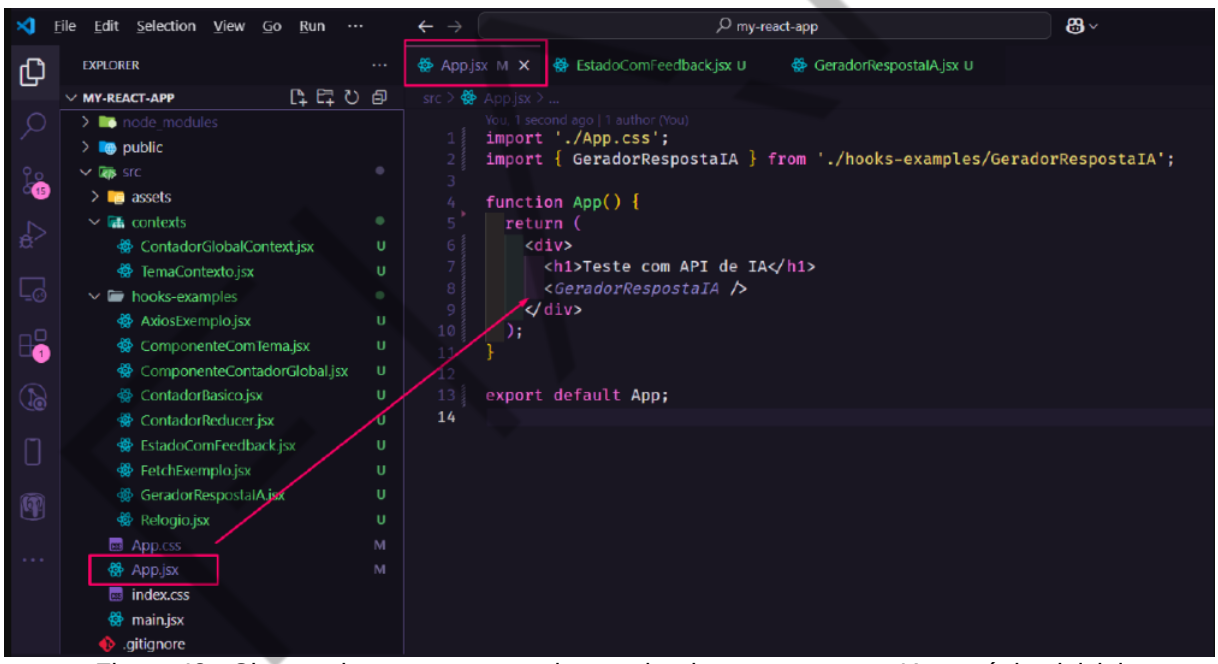


Figura 43 - Chamando o componente de gerador de respostas com IA na página inicial
Fonte: Elaborado pelo autor (2025)

Para que funcione corretamente, é necessário:

- Criar uma conta no site da OpenAI.
- Gerar uma chave de API pessoal.
- Substituir a string SUA_CHAVE_OPENAI no código acima por sua chave válida.

Passo a passo para obter a chave da API da OpenAI:

- **Passo 1 –** Acesse o site oficial da OpenAI: <https://platform.openai.com>.
- **Passo 2 –** Crie uma conta (ou faça login):
  - Você pode se cadastrar com um e-mail ou usando contas do Google/GitHub.
  - É gratuito para começar: a OpenAI costuma oferecer um crédito inicial gratuito em novas contas.
- **Passo 3 –** Acesse o Painel da API:
  - Depois de logado, vá até a aba **API keys** ou clique direto no link: <https://platform.openai.com/account/api-keys>.
- **Passo 4 –** Clique em “+ Create new secret key”:
  - Dê um nome descritivo, como react-teste-ia.
  - Copie a chave gerada **imediatamente**, pois ela será exibida apenas uma vez.
- **Passo 5 –** Use a chave no seu projeto:
  - No seu código React, substitua a string: Authorization: 'Bearer SUA_CHAVE_OPENAI' por: Authorization: 'Bearer sk-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX'

Atenção: nunca compartilhe sua chave de API publicamente (GitHub, vídeos ou grupos).

O que fazer se aparecer o erro 429 – Too Many Requests:

Esse erro significa que a API da OpenAI **recusou a requisição**, pois o número de chamadas feitas **excedeu o limite permitido** para a sua conta naquele momento. Isso pode acontecer por alguns motivos:

- Você clicou várias vezes no botão "Enviar" em sequência.
- Sua conta gratuita da OpenAI já **atingiu o limite de uso diário ou por minuto**.

- A mesma chave de API está sendo usada em **vários dispositivos ou navegadores** ao mesmo tempo.

Como resolver:

- Aguarde cerca de 1 minuto e tente novamente.
- Evite apertar o botão repetidamente enquanto a resposta não chega.
- Se o erro persistir:
  - Acesse <https://platform.openai.com/account/usage>.
  - Verifique se seu plano atingiu o limite de uso (tokens ou chamadas).
- Caso tenha compartilhado sua chave, gere uma nova em: <https://platform.openai.com/account/api-keys>.

Dica de segurança:

Nunca compartilhe publicamente sua chave da OpenAI. Se estiver trabalhando em equipe ou com versionamento (como GitHub), armazene a chave em um arquivo .env e use import.meta.env.VITE_OPENAI_KEY no código. Se quiser, podemos mostrar como fazer isso no projeto com Vite.

EIAD

5 AUTENTICAÇÃO DE USUÁRIOS COM JWT E CONTEXT API

5.1 O que é o JSON Web Token (JWT)?

O **JWT (JSON Web Token)** é um formato compacto e seguro para transmitir informações entre sistemas como um objeto JSON assinado digitalmente. Ele é amplamente utilizado para autenticação em APIs modernas. Após o login, o backend gera um token e o cliente guarda esse token para provar que está autenticado em futuras requisições.

5.2 Armazenamento e verificação de tokens

Tokens JWT geralmente são armazenados no **localStorage** ou **AsyncStorage** (em apps mobile). No React Web, usamos o localStorage para manter o usuário autenticado entre as atualizações da página. Antes de acessar áreas protegidas, o frontend verifica se o token existe e se de fato é válido.

5.3 Implementando autenticação com Context API

Crie um arquivo em **src/contexts/AuthContext.jsx**:

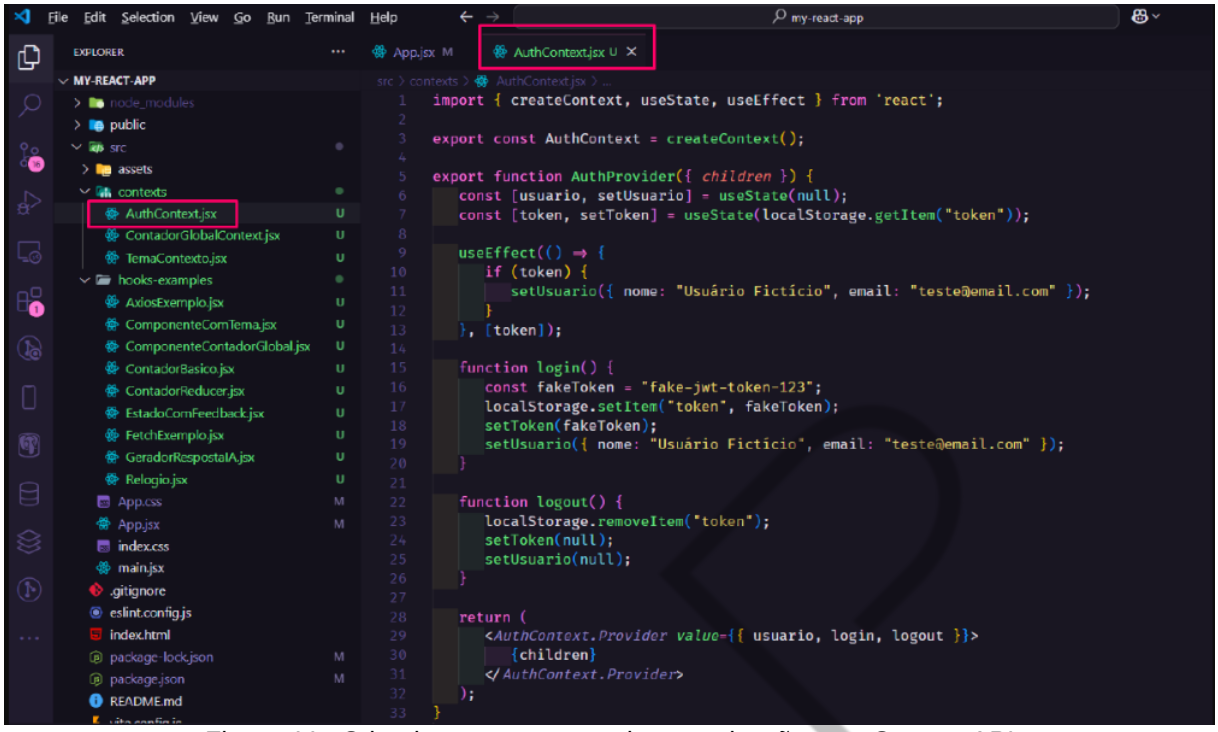


Figura 44 - Criando o componente de autenticação com Context API
Fonte: Elaborado pelo autor (2025)

A seguir, veja as explicações dos trechos do código.

Importação de recursos:

```
import { createContext, useState, useEffect } from 'react';
```

Código-fonte 29 - Importação de hooks e context do React
Fonte: Elaborado pelo autor (2025)

Aqui, estamos importando as ferramentas do **React** que serão usadas para:

- Criar um Contexto global de autenticação (**createContext**).
- Gerenciar estados (**useState**).
- Executar um efeito após o carregamento ou atualização do componente (**useEffect**).

Criação do contexto de autenticação:

```
export const AuthContext = createContext();
```

Código-fonte 30 - Exportando contexto de autenticação
Fonte: Elaborado pelo autor (2025)

Essa linha cria o contexto que será compartilhado entre os componentes que desejarem acessar informações de autenticação, como o nome do usuário, se está logado e as funções de login/logout.

Componente AuthProvider:

```
export function AuthProvider({ children }) {
```

Código-fonte 31 - Passando a autenticação entre componentes
Fonte: Elaborado pelo autor (2025)

Esse componente envolve toda a aplicação e fornece o estado de autenticação para os componentes filhos. Ele é utilizado no App.jsx como um “embrulho global” da aplicação.

Gerenciamento de estado local:

```
    const [usuario, setUsuario] = useState(null);
    const [token, setToken] =
useState(localStorage.getItem("token"));
```

Código-fonte 32 - Gerenciamento de estado iniciado como nulo
Fonte: Elaborado pelo autor (2025)

- **usuario:** guarda as informações do usuário autenticado.
- **token:** tenta recuperar o token armazenado no navegador via localStorage.

Isso simula a persistência de login mesmo que a página seja recarregada.

```
    useEffect para login automático com token salvo
    useEffect(() => {
      if (token) {
         setUsuario({ nome: "Usuário Fictício", email:
"teste@email.com" });
      }
    }, [token]);
```

Código-fonte 33 - Simulando login se tiver token
Fonte: Elaborado pelo autor (2025)

Este **useEffect** é executado sempre que o token mudar. Se houver um token válido (mesmo fake), ele preenche os dados do usuário.

Essa função simula um login:

```
      Função login()
      function login() {
         const fakeToken = "fake-jwt-token-123";
         localStorage.setItem("token", fakeToken);
         setToken(fakeToken);
         setUsuario({ nome: "Usuário Fictício", email:
"teste@email.com" });
      }
```

Código-fonte 34 - Simulando token para login
Fonte: Elaborado pelo autor (2025)

Em que:

- Cria um token fictício.
- Salva esse token no localStorage.
- Atualiza os estados token e usuario.

Essa função simula o logout:

```
      Função logout()
      function logout() {
         localStorage.removeItem("token");
         setToken(null);
         setUsuario(null);
      }
```

Código-fonte 35 - Removendo token ao sair
Fonte: Elaborado pelo autor (2025)

Em que:

- Remove o token do navegador.
- Limpa o usuário e token do estado.

Compartilhando os dados com o Context:

```
      return (
         <AuthContext.Provider value={{ usuario, login, logout
}}>
            {children}
         </AuthContext.Provider>
      );
```

Código-fonte 36 - Passando dados via context entre componentes
Fonte: Elaborado pelo autor (2025)

Esta parte torna usuário, login e logout disponíveis para qualquer componente da aplicação que usar useContext(AuthContext).

Agora, crie o componente **src/hooks-examples/PainelLogin.jsx**:

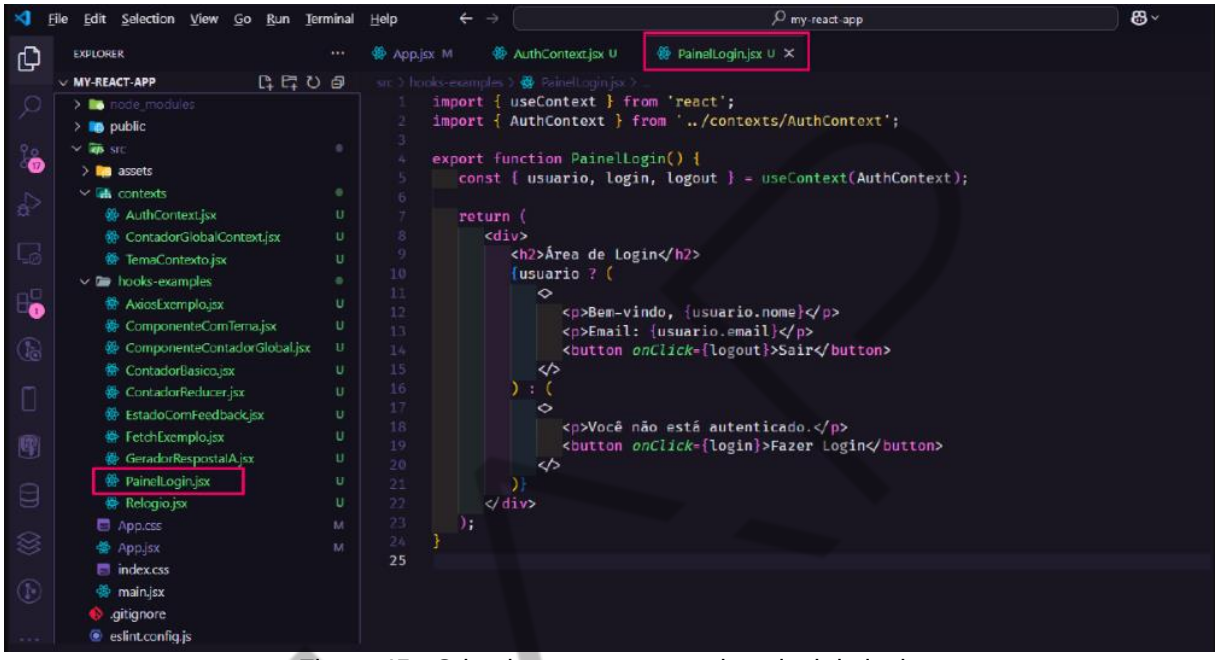


Figura 45 - Criando o componente de painel de login
Fonte: Elaborado pelo autor (2025)

Importações necessárias:

```
import { useContext } from 'react';
import { AuthContext } from '../contexts/AuthContext';
```

Código-fonte 37 - Importação de autenticação do React
Fonte: Elaborado pelo autor (2025)

- useContext é um hook do React que permite acessar qualquer contexto criado com createContext.
- AuthContext é o contexto de autenticação criado anteriormente, que fornece usuario, login e logout.

Acessando o contexto:

```
const { usuario, login, logout } = useContext(AuthContext);
```

Código-fonte 38 - Iniciando sessão ao receber a autenticação
Fonte: Elaborado pelo autor (2025)

Com essa linha, o componente recebe:

- **usuario:** objeto com os dados do usuário autenticado (ou null se não estiver logado).
- **login:** função para simular login e gerar um token fake.
- **logout:** função para simular logout e remover o token.

Renderização condicional baseada na autenticação:

```
return (
  <div>
    <h2>Área de Login</h2>
    {usuario ? (
      <>
        <p>Bem-vindo, {usuario.nome}</p>
        <p>Email: {usuario.email}</p>
        <button onClick={logout}>Sair</button>
      </>
    ) : (
      <>
        <p>Você não está autenticado.</p>
        <button onClick={login}>Fazer Login</button>
      </>
    )}
  </div>
);
```

Código-fonte 39 - Renderização condicional baseada na autenticação
Fonte: Elaborado pelo autor (2025)

Essa estrutura usa o operador ternário para mostrar conteúdos diferentes, dependendo se usuário está definido.

- **Se o usuário estiver autenticado:**
  - Exibe o nome e o e-mail do usuário.
  - Mostra um botão para fazer logout.
- **Se o usuário não estiver autenticado:**
  - Mostra uma mensagem de acesso negado.
  - Exibe um botão que executa login().

5.4 Proteção de rotas e controle de acesso

Com a Context já funcionando, podemos proteger componentes específicos com base no estado de autenticação.

Crie **src/hooks-examples/AreaRestrita.jsx**:

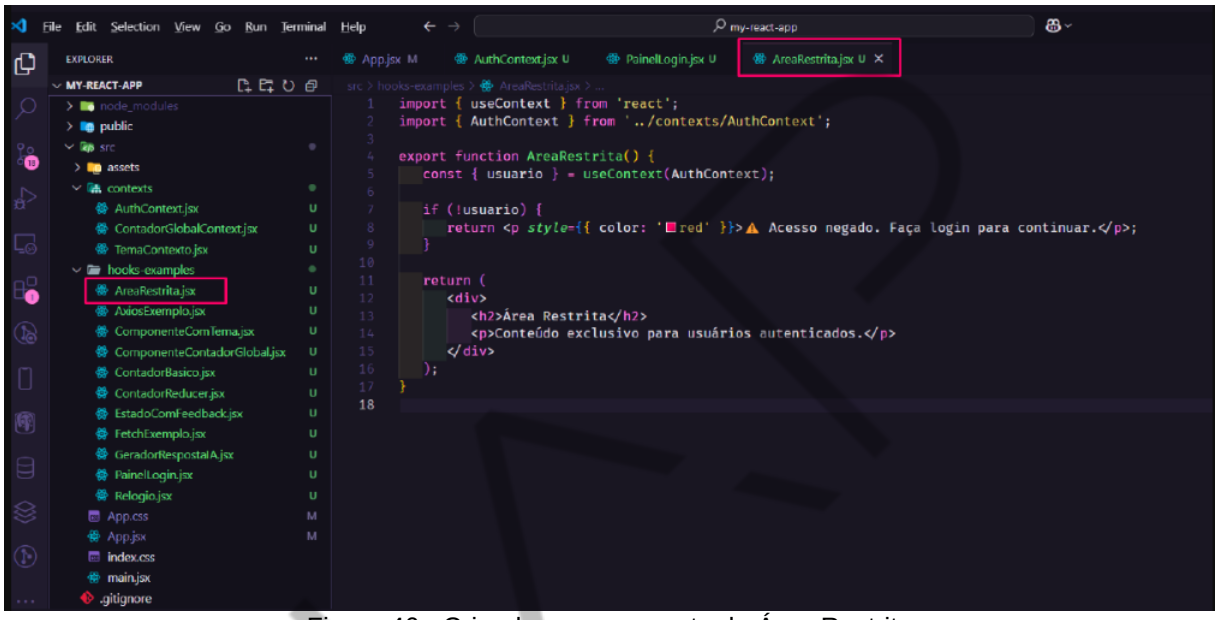


Figura 46 - Criando o componente de Área Restrita
Fonte: Elaborado pelo autor (2025)

Importações necessárias:

```
import { useContext } from 'react';
import { AuthContext } from '../contexts/AuthContext';
```

Código-fonte 40 - Importação da autenticação
Fonte: Elaborado pelo autor (2025)

- Importamos o useContext para acessar os dados do contexto.
- AuthContext é o contexto que gerencia a autenticação do usuário.

Acessando os dados do contexto:

```
const { usuario } = useContext(AuthContext);
```

Código-fonte 41 - Recebendo dados após a autenticação
Fonte: Elaborado pelo autor (2025)

Aqui, estamos recuperando apenas o dado usuario, pois nesse componente só precisamos saber se há alguém autenticado ou não.

Proteção de acesso com verificação condicional:

```
    if (!usuario) {
      return <p style={{ color: 'red' }}>⚠ Acesso negado. Faça login para continuar.</p>;
    }
```

Código-fonte 42 - Feedback de falha de login
Fonte: Elaborado pelo autor (2025)

Antes de renderizar qualquer conteúdo protegido, precisamos verificar se o usuário está definido.

Se **não estiver logado**, o componente retorna uma mensagem de acesso negado e **não renderiza o restante da tela**.

Conteúdo liberado apenas para usuários autenticados:

```
    return (
      <div>
        <h2>Área Restrita</h2>
        <p>Conteúdo exclusivo para usuários autenticados.</p>
      </div>
    );
```

Código-fonte 43 - Exibição de conteúdo após tela autenticada
Fonte: Elaborado pelo autor (2025)

Caso o usuário esteja presente no contexto, a área protegida é exibida normalmente.

Isso simula a lógica de rotas protegidas em um app real, sendo a base para proteger páginas inteiras.

5.5 Simulação de backend com mock

Neste exemplo, usamos um “falso login” sem API real. Em aulas futuras, você pode substituir a função login() por uma requisição real com axios:

```
    axios.post("https://api.suaapifalsa.com/login", { email, senha })
      .then(res => localStorage.setItem("token", res.data.token));
```

Código-fonte 44 - Mock para login
Fonte: Elaborado pelo autor (2025)

Agora, teste no App.jsx:

Abra o **src/App.jsx** e substitua o conteúdo:

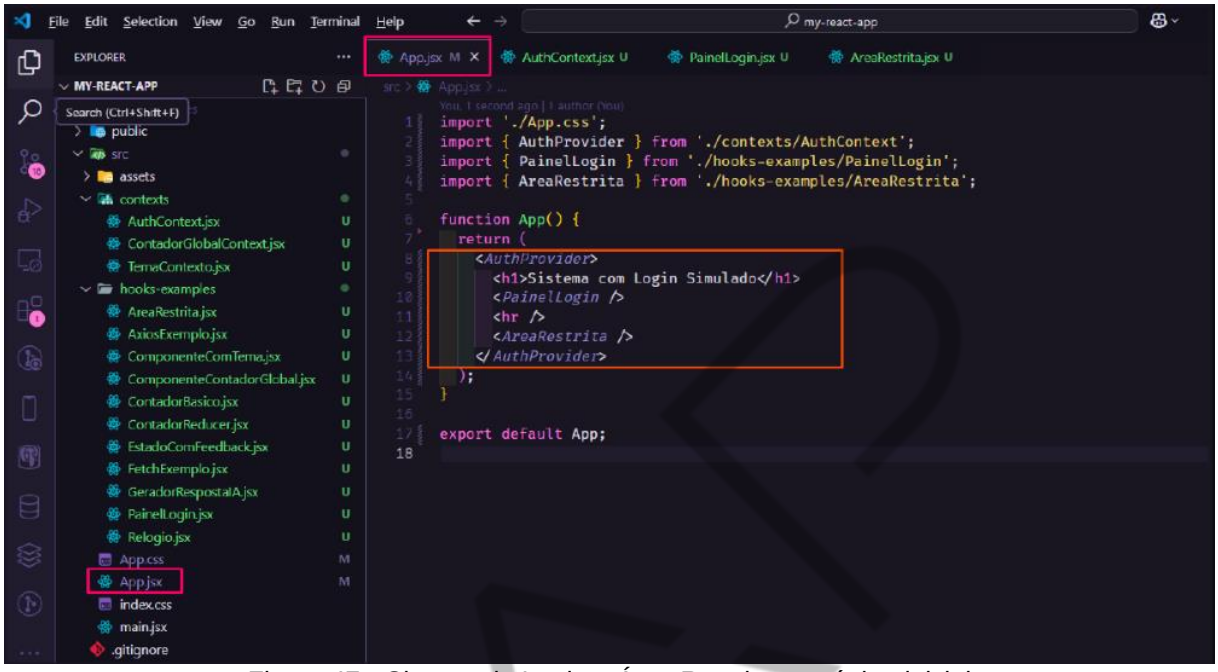


Figura 47 - Chamando Login e Área Restrita na página inicial
Fonte: Elaborado pelo autor (2025)

Envolvendo tudo com AuthProvider:

```
<AuthProvider>
  {/* componentes aqui dentro têm acesso ao contexto */}
</AuthProvider>
```

Código-fonte 45 - Exibição de conteúdo após autenticado
Fonte: Elaborado pelo autor (2025)

- O AuthProvider é o **componente de contexto** que criamos anteriormente.
- Ele encapsula toda a lógica de login, logout e gerenciamento de token simulado.
- Tudo que estiver dentro desse Provider **terá acesso à autenticação via useContext(AuthContext)**.

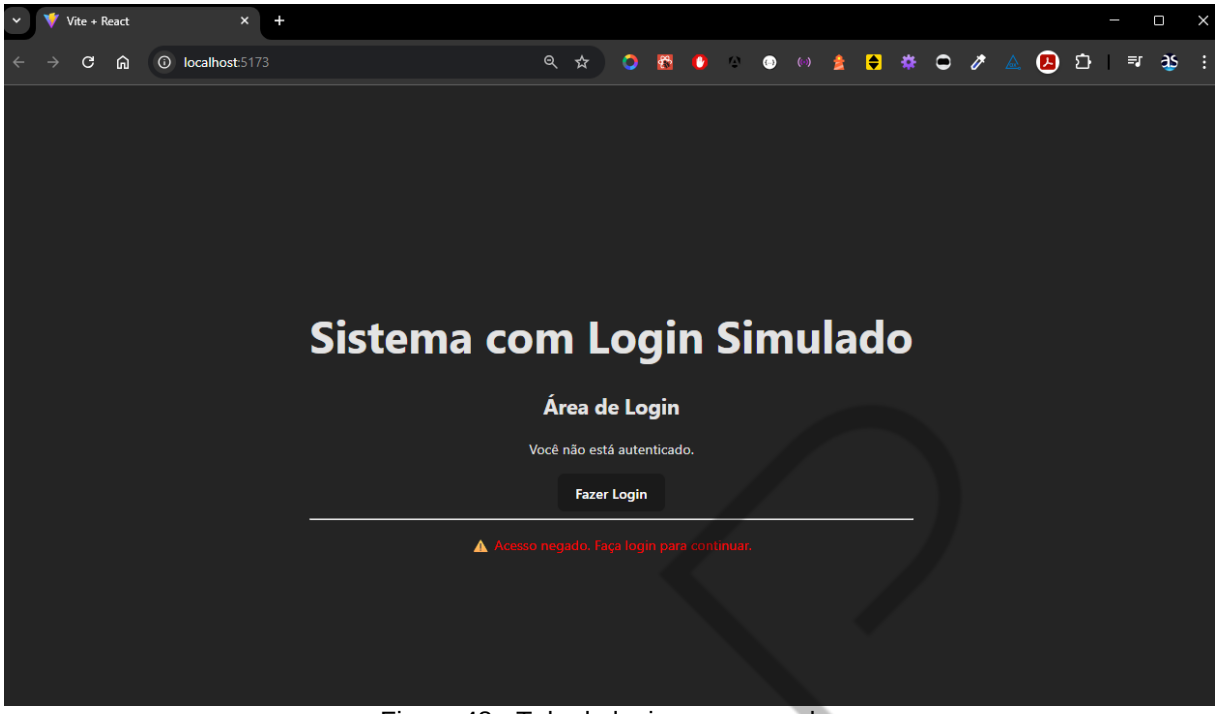


Figura 48 - Tela de login no navegador
Fonte: Elaborado pelo autor (2025)

REFERÊNCIAS

PONTES, Guilherme**. Progressive Web Apps:** construa aplicações progressivas com React. São Paulo: Casa do Código, 2018. 441 p.

FIAP